Expert Systems With Applications 297 (2026) 129037



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





A comparative study of word embedding techniques for classification of star ratings ⋆

Hesham Abdelmotaleb *, Craig Mcneile , Małgorzata Wojtyś

Centre for Mathematical Sciences,University of Plymouth, Drake Circus, Plymouth, PL4 8AA, Devon, United Kingdom

ARTICLE INFO

Keywords:
Text classification
Word embeddings
Feature extraction
Short texts
Telecom data

ABSTRACT

Telecom services are at the core of today's societies' everyday needs. The availability of numerous online forums and discussion platforms enables telecom providers to improve their services by exploring the views of their customers to learn about common problems that customers face. Natural Language Processing (NLP) tools can be used to process the free text collected.

One way of working with such data is to represent text as numerical vectors using one of many word embedding models based on neural networks. This research uses a novel dataset of telecom customers' reviews to perform an extensive comparative study showing how different word embedding algorithms can affect the text classification process. A variety of state-of-the-art word embedding techniques are considered, including BERT, Word2Vec, FastText, and Doc2Vec. Several PCA-based approaches are explored for feature engineering. Moreover, the energy consumption used by the different word embeddings is investigated. The findings show that BERT combined with PCA lead to consistently better text classifiers in terms of precision, recall and F1-Score, particularly for more challenging classification tasks. Moreover, our proposed PCA approach of combining word vectors using the first principal component shows clear advantages in performance over the traditional approach of taking the average.

1. Introduction

There is potentially a lot of useful insight into customer views of a company and its products from free text in online discussion forums and social media posts. For example, persistent negative online views of a product may result in future reduced revenue from customers no longer buying products or stopping paying for the subscription service (customer churn). Hence, it is important to be able to process free text using Natural Language Processing (NLP) tools. This could include summarizing a large chunk of free text or incorporating features from free text into classifiers.

The goal of this research is to investigate techniques to build classifiers of customer star ratings from the customer's free text. For the dataset, we use short free texts about telecom companies written by customers on online platforms. Broadband and mobile connectivity are crucial to the running of the economy. In principle, it is easy for customers to switch broadband providers, so monitoring customer feedback and building classifiers to predict customer churn is important for the profitability of a telecom company. We use open data to carefully study the performance of critical aspects of classification algorithms.

In this era of big data, it is strategic to study the performance of machine learning algorithms as the sizes of datasets increase (Upadhyay et al., 2020). Also, due to concerns about the climate crisis, an increasingly important aspect of data science computations is minimizing the energy consumed in training and using machine learning models (Lottick et al., 2019; Mehlin et al., 2023). Hence, we measure energy consumption and CO2 emissions in our benchmarks.

In the empirical study, we consider the classification of star ratings based on telecom customers' opinions written online. Specifically, we explore the use of word embeddings and feature extraction and perform a comparative study of a wide range of algorithms in the context of a classification task in the telecom data field, to find the most effective approach that is time-efficient and accurate. Abdelmotaleb et al. (2023) considered similar short texts in the telecom industry, but focused on unsupervised topic modeling.

Several previous studies explored building classifiers to predict star ratings, highlighting the importance of this task in many applications, including sentiment analysis, customer churn prediction and text classification. Binder et al. (2019) explained the idea of stars in the field of online customer reviews, while applying regression to model the

⋆ This document is the results of the research project funded by the University of Plymouth.

* Corresponding author.
E-mail addresses: hesham.abdelmotaleb@plymouth.ac.uk (H. Abdelmotaleb), craig.mcneile@plymouth.ac.uk (C. Mcneile), malgorzata.wojtys@plymouth.ac.uk (M. Wojtyś).

https://doi.org/10.1016/j.eswa.2025.129037
Received 21 May 2025; Received in revised form 5 July 2025; Accepted 13 July 2025
Available online 29 July 2025
0957-4174/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

rating value. After using a two-step approach to assess the results, an underlying model for preferences on a continuous scale and a nonlinear transformation of the underlying preferences onto the rating scale were employed. Monett and Stolte (2016) focused on predicting the review's star rating using annotated mobile app reviews collected from Google Play Store, by using sentiment analysis as a feature extraction model and then regression algorithms to predict each review's rating. In a different study, Saumya et al. (2018) researched how ranking online consumer reviews can be measured. The authors extracted 17 features from three types of datasets, combining linguistic, numerical and NLP features. The results showed that Random Forest generally performed better than Naive Bayes and SVM.

The focus of this research is to compare a broad spectrum of popular word embedding methods in the context of building a classification model for star ratings based on short texts written by customers. In the existing literature, many studies have been conducted that compare selected word embedding algorithms for various tasks. For example, Zhang et al. (2025) investigated two methods: TF-IDF (combined with a Transformer) and BERT for analyzing online review data in the tourism industry. For another example, Wang et al. (2018) evaluated word embeddings that were trained from different corpora (clinical notes, biomedical literature, Wikipedia and news) for biomedical NLP applications. Sabbeh and Fasihuddin (2023) compared BERT, AraBERT, FastText, Word2Vec, AraVec and GloVe for Arabic sentiment classification, using five datasets with different dialects and domains. The authors found that AraBERT showed better performance amongst other word embeddings and that BiLSTM classifier achieved higher accuracy than CNN for three datasets, but CNN performed better on a smaller dataset. Wang et al. (2020) studied the classic and contextualized word embeddings for text classification comparing Word2Vec, GloVe and FastText to ELMo and BERT, using CNN and BiLSTM as classifiers. The results indicated that CNN outperformed BiLSTM for document classification, whereas BiLSTM was more effective for sentence-level sentiment analysis. BERT generally outperformed ELMo, particularly for longer documents and the study suggests using CNN with classic embeddings for document classification and BiLSTM with contextualized embeddings for sentence-level classification. Our work addresses the gap in comparative performance evaluations for short-text classification using real-world telecom data. Also it contributes a new benchmark for combining word embeddings with PCA, integrating classifier performance, processing time, and environmental costs.

Feature engineering and dimension reduction are important and challenging aspects of model building when using word embeddings because of the high dimensionality of the feature space. Okkalioglu (2024) proposed a feature selection method aiming at removing neutral terms based on measuring the distance between a term and its neutral counterpart. Occhipinti et al. (2022) tested twelve machine learning models for spam filtering text classification, focusing on optimizing feature sizes and hyperparameters in the preprocessing stage. Lebret and Collobert (2014) explored the use of Principal Component Analysis (PCA) with word embeddings, specifically applying Hellinger PCA to a word co-occurrence matrix. The authors found that PCA could produce embeddings comparable to those derived from deep learning models while being much faster and computationally efficient. The resulting embeddings performed well on NLP tasks such as Named Entity Recognition and sentiment analysis. In a different study, Gupta et al. (2019) explored how to improve word embeddings using Kernel Principal Component Analysis (KPCA). The authors demonstrated that applying KPCA to word similarity matrices improved its performance, providing a better initialization for word embedding models such as Word2Vec and FastText, especially for small datasets and infrequent words. Additionally, KPCA embeddings lead to faster convergence reducing training time while maintaining high-quality representations. Moreover, Eisa et al. (2018) investigated the use of PCA for feature extraction with TF-IDF in the context of text classification. Random Forest, SVM, Decision Tree and KNN classifiers were considered. The authors found that applying PCA to TF-IDF text representation improved classifier performance. Among the classifiers tested, Random Forest showed the most significant performance gain with PCA. Therefore, overall, PCA is a promising method for enhancing text classification by optimizing feature representations.

The aim of this research is to study and compare a broad range of state-of-the-art word embeddings employed for a task of short texts classification to distinguish between customers' satisfaction levels. We investigate the following word embedding methods: Word2Vec, FastText, BERT and Doc2Vec. For Word2Vec and FastText. Two ways of combining the word vectors are explored: using the average and the first principal component. PCA-based dimension reductions for BERT and Doc2Vec are also studied. Moreover, TF-IDF text representations are included for comparison. We apply supervised classification algorithms: Logistic Regression, Random Forest, Gradient Boosting classifier, Stochastic Gradient Descent, Decision Tree, Support Vector Machine and K-Nearest Neighbours (KNN). Their performance is measured using precision, recall and F1-Score (Powers, 2011). Moreover, energy consumption and CO_2 emissions are measured for word embeddings models and analyzed in a comparative study. The methods are applied to data collected from online comments of telecommunications customers and several scenarios are considered including binary classification and multiclass classification of the number of stars. All the datasets are original for this research and were obtained by scraping the data from online forums. By investigating several popular word embeddings, various feature extraction methods and a wide range of classifiers in one study, a comprehensive comparison is achieved, making the findings highly relevant for practical applications in the studied domain.

This study makes a significant contribution to existing knowledge in several ways. While our methodology is based on using existing models, its novelty lies in the unified and energy-conscious evaluation across multiple word embedding algorithms and classification models using a real telecom dataset. Unlike prior works, our study also reports on computational efficiency and scalability across multiple setups which, in the era of global warming and striving for net-zero systems, is a timely and highly relevant contribution to the field of sustainable computing. Moreover, a new benchmark for combining word embeddings with PCA, integrating classifier performance, processing time, and environmental costs is established. To the best of authors' knowledge, a comparative study of several PCA-based feature engineering methods as defined in this paper has not been performed before for short texts data and various word embedding models. In particular, the proposed modification of the popular averaging of word embedding vectors by using the weighted mean obtained from the first Principal Component is demonstrated to have superior performance while remaining time- and computationally efficient as well as relatively straightforward in implementation, which provides valuable evidence-based guidelines for practitioners.

The results of this study show that, for Word2Vec and FastText, our proposed PCA approach of combining word vectors using the first principal component outperforms the traditional approach of taking the average. Moreover, BERT combined with PCA-based dimensionality reduction generally leads to the best performance among all other considered word embeddings, especially for more challenging multiclass tasks. Among classifiers, Gradient Boosting and KNN demonstrated the highest F1-Scores.

The paper is organized as follows. Section 2 discusses the word embeddings and feature engineering methods to be applied to the datasets and describes the study design. Section 3 discusses the results, including the performance, time and energy consumption. In Section 4 we report the conclusions from the study.

2. Methodology and study design

This section describes the building blocks of the methods studied in this research. First, a general background of the word embedding models Word2Vec, Doc2Vec, FastText and BERT is provided. Next, the proposed feature engineering methods to be applied on the word

embedding vectors are introduced. These consist of leveraging the first Principal Component for Word2Vec and FastText vectors as well as applying PCA to lower the dimensionality of BERT and Doc2Vec vectors. Finally, the pipeline of the empirical study is presented, including in particular the process of sampling the datasets of short texts and the choice of metrics used to evaluate and compare the considered methods.

2.1. Word embedding models

This subsection provides a general introduction to the word embedding models considered in this study: Word2Vec, Doc2Vec, FastText and BERT. The choice of word embeddings selected for this study is motivated by their popularity in practical applications, as well as the aim to cover a range of methods including earlier models, such as Word2Vec, and more recent and advanced ones, such as BERT, as well as methods that employ various approaches to representing texts, such as word- and document vectors.

Word Embeddings (WEs) are a type of NLP techniques that represent words as numerical vectors. They have been shown to be useful and effective for many NLP tasks, including text classification, machine translation and sentiment analysis (Oscar Deho et al., 2018). Once the WEs have been trained, they can be used as input to Machine Learning models to classify customer service scripts into different categories. Then, their outputs can help to develop and increase customer service efficiency and to find common customer issues. This study examines the following popular WEs:

1. Word2Vec (Mikolov et al., 2013), which is based on employing a neural network and involves predicting the context of a word in a sentence (Kottur et al., 2016). Word2Vec generates a vector for each word in a document.
2. Doc2Vec, which is an extension of Word2Vec that represents entire documents as numerical vectors. Doc2Vec also trains WEs by predicting the context of a word in a document (Shao et al., 2018). Doc2Vec embeddings were trained using the Distributed Memory (DM) model with 300 dimensions and a context window of 5. For example, a document such as “The broadband service was poor and inconsistent” would be transformed into a single 300-dimensional vector representing its overall semantics. This is the main difference between Word2Vec and Doc2Vec, as Word2Vec would represent each word individually.
3. FastText is another extension of Word2Vec that can handle out-of-vocabulary words and was developed by Facebook AI Research. FastText algorithm trains WEs by representing each word as a bag of character n-grams (Lakmal et al., 2020), producing 300-dimensional vector for each word (Yao et al., 2020). FastText supports both Skip-gram and CBOW structures.
4. BERT (Bidirectional Encoder Representations from Transformers) (Chang et al., 2019) is a deep learning algorithm that is typically trained on a large corpus of text. BERT can be used not only to create WEs but also other NLP tasks, such as machine translation and question answering (Wang et al., 2019). BERT considers the context of each word in two directions, as it uses a bidirectional transformer algorithm that considers the long-range dependencies between all words in any sentence. This makes BERT able to understand the word embedding values better compared with traditional word embedding techniques like Word2Vec and GloVe (Pennington et al., 2014). BERT was pre-trained on 3.3 billion tokens (Clark et al., 2019).

In addition, the traditional TF-IDF (Sammut & Webb, 2010) text representations are applied and included in the study for comparison.

2.2. Feature engineering

In this subsection, the proposed feature engineering methods to be applied on the four types of word embedding vectors, described in Section 2.1, are explained in detail. Several scenarios are considered. For Word2Vec and FastText, the first Principal Component is leveraged to combine the word embedding vectors. In two other scenarios, PCA is applied in a different manner to lower the dimensionality of averaged BERT and of Doc2Vec vectors, respectively. The choice of PCA for this study is motivated by its computational speed, simplicity, and its sound theoretical background for its ability to capture the maximum amount of information using linear combinations of features, which is a natural genrelization of taking an average (Jolliffe, 2002).

All WE algorithms are trained to extract the embedded meaning of the text and represent it as a numerical vector for either the word or sentence or the whole document. Some methods, including Word2Vec, GloVe, FastText and BERT extract the embedded vector for each word, while others such as Doc2Vec can extract one embedded vector for the entire document. Using a WE algorithm for the entire text document makes the extracted vector potentially ready for machine learning models, as each sample row has one numerical vector of attributes. However, using WE algorithms that extract word-by-word vectors creates a challenge as to a suitable mathematical method to combine the vectors and create a lower-dimensional set of features for the whole document.

A frequently applied approach is to obtain the average vector over all terms in a document. So, if $v_1, \dots, v_{n_d} \in R^m$ are term vectors for document d, then $x_d = (v_1 + \dots + v_{n_d})/n_d$ represents its feature vector, where n_d is the number of terms in document d and m is the dimension of the word embedding vectors. Here, each term contributes equally into the resulting vector x_d.

Another possible approach is to employ Principal Component Analysis (PCA) (Anderson, 2003) as a dimension reduction and feature extraction method. We consider several PCA-based ways to represent features for various word embeddings, as detailed below.

2.2.1. PCA for Word2Vec and FastText

Word2Vec and FastText produce separate embedding vectors for each term in a document. In this case, the PCA method will be used as a way of modifying term-related weights in the average vector x_d. Namely, the feature vector for document d will be in the form $w_1 v_1 + \dots + w_{n_d} v_{n_d}$, where $w_1, \dots, w_{n_d}$ are weights related to terms in the document. The weights are obtained from the PCA method by taking the first principal component. Therefore, this PCA-based feature representation is obtained in two steps. First, for each document d, term vectors $v_1, \dots, v_{n_d} \in R^m$ are obtained and placed in a matrix $A_d = [v_1, \dots, v_{n_d}] \in R^{m \times n_d}$ with each column corresponding to one term. The second step is to apply the PCA to this matrix, then select the first principal component, which, mathematically, is a weighted average of the column vectors. Note that here the PCA method is performed separately for each document d.

To determine a better approach to feature engineering for Word2Vec and FastText, we compare taking the arithmetic mean of vectors, which is the more popular practice, to applying PCA which attempts to maximise the amount of information extracted from the terms’ vectors. This will produce four text representations, which we denote in this study as W2V-Average, W2V-PCA, FT-Average, FT-PCA, for Word2Vec and FastText word embeddings, respectively. For Word2Vec, the dimension m of word embedding vectors and therefore the dimension of feature vectors x_d, is equal to 100 and for FastText $m = 300$.

2.2.2. PCA for BERT

For BERT, we consider another PCA-based feature extraction approach consisting of the following steps: (1) apply the mean pooling to the individual term vectors to obtain one vector representing each document; (2) use PCA to reduce the dimension of the document vectors resulting from the above step.

Here, the PCA technique is applied to reduce the dimension of the averaged vectors for all documents together, rather than to combine the vectors of individual terms separately for each document. As a result, the dimension of BERT embeddings is reduced from 384 to 50.

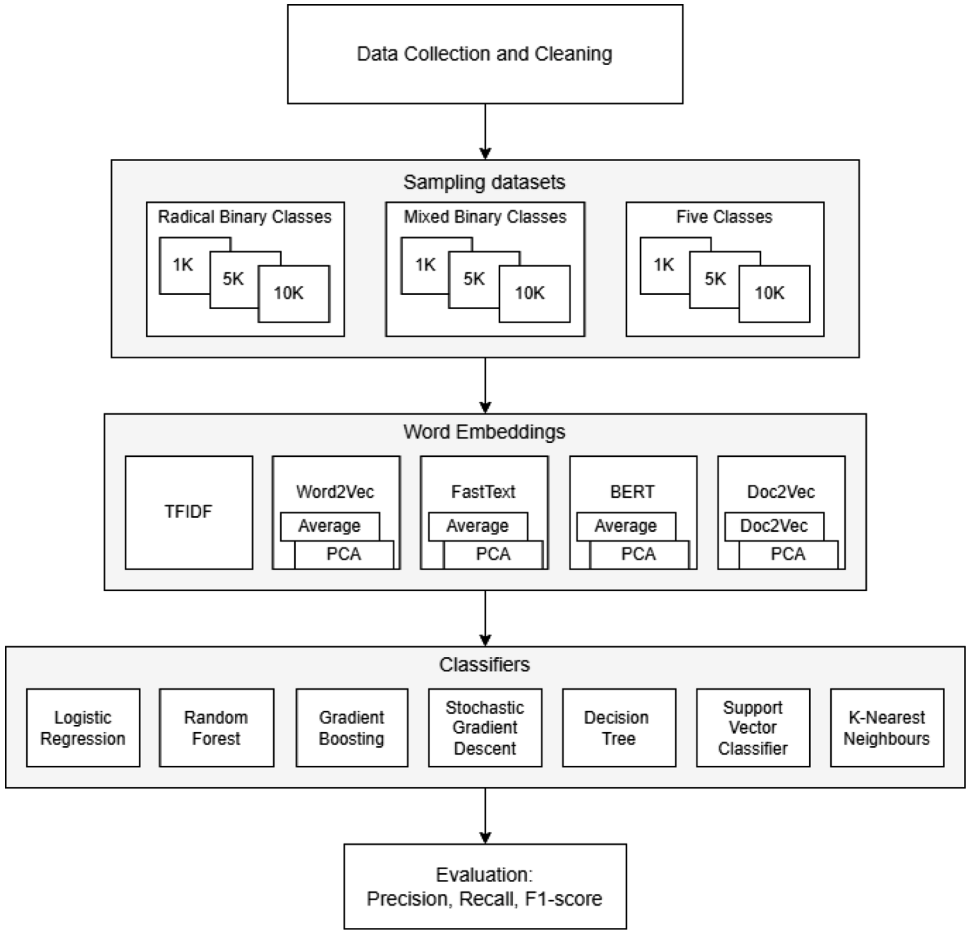


Fig. 1. The pipeline of the empirical study as explained in Section 2.3.

Two text representations are compared: BERT-Average and BERT-PCA, where the BERT-Average dataset is produced using the function `mean_pooling` from `transformers` library, which computes the weighted averages of 384-dimensional word embedding vectors for each document and BERT-PCA is produced by applying PCA dimensionality reduction to the BERT-Average vectors as described above.

2.2.3. PCA for Doc2Vec

We also invetigate an application of PCA to Doc2Vec document vectors, reducing the features' dimension from 300 to 100, to examine its usefulness in comparison to taking the entire document vector. The feature representations obtained from the above procedures are denoted in this study as Doc2Vec and Doc2Vec-PCA, respectively.

2.3. Study design

In this subsection, the pipeline of the empirical study is presented, including in particular the process of sampling the datasets of short texts and the choice of metrics used to evaluate and compare the considered methods.

The empirical study is organized in the following general steps: (1) creating different datasets, (2) applying feature extraction techniques with word embeddings, (3) applying different classifiers to the obtained features and (4) comparing time consumed and performance metrics for the resulting features and classifiers. Fig. 1 shows the pipeline of the study and the details are described in the following paragraphs.

2.3.1. Step 1: Creating the different datasets

The starting dataset consisted of online customer reviews for 29 telecommunication brands in the UK, scraped from the Trustpilot platform and contained 367,000 reviews. Each review was combined with a star rating, ranging between 1 and 5 stars, given by the reviewer to produce a labeled dataset suitable to be used to build a text classifier. Text cleaning tools were applied to the dataset to remove all unwanted characters, make all letters small and remove non-English letters and words.

Next, three types of datasets were created[1] using random sampling:

- **Radical Binary Data** consisting of reviews with 1-star, labeled as "Bad" and 5-star, labeled as "Good", in equal proportions. Three sets of sample size 1000, 5000 and 10,000, respectively, were randomly selected.
- **Mixed Binary Data** is a dataset with two classes "Bad" and "Good", where the class "Bad" consists of a mixture of 1-star and 2-star reviews. The class "Good" consists of reviews that have a ranking value of 4 or 5 stars. The aim of studying this dataset is to introduce less obvious patterns in reviews for a more challenging classification task. Three sets of size 1000, 5000 and 10,000, respectively, were randomly selected.
- **Multi-Class Data** with five classes was created by selecting an equal number of reviews for each star ranking, from 1 to 5. This set of data is used to apply multiclass classification and to explore the performance of word embeddings and classifiers in a more challenging

[1] The datasets are available at https://github.com/HeshamAsem/WEDataset/

scenario. Three sets of size 1000, 5000 and 10,000, respectively, were randomly selected.

The created datasets consisted of two columns: Text and Type, where the Text column contained the cleaned customer's review and the Type column contained either a binary class label ("Good" or "Bad") in the case of the binary class datasets or the qualitative label for the number of stars ranging from 1-star to 5-stars in the case of the multiclass datasets.

2.3.2. Step 2: Feature extraction techniques

Feature extraction algorithms convert the raw text into represented numerical values, that aim to reflect its meaning. In this study, nine word-embedding-based techniques were explored, as described in Section 2.2: W2V-Average, W2V-PCA, FT-Average, FT-PCA, BERT-Average, BERT-PCA, Doc2Vec, Doc2Vec-PCA and TF-IDF.

These word embedding techniques were applied to the nine datasets producing different text representations of each one of them, that will be used with classifiers.

2.3.3. Step 3: Classifiers

We employed the following seven classification algorithms for each dataset to predict the star rating based on the customer's written text: Logistic Regression, Random Forest, Gradient Boosting, Stochastic Gradient Descent, Decision Tree, Support Vector Machine and K-Nearest Neighbours (KNN). This choice of classifiers covers a range of popular and well established methods including simpler parametric models, such as Logistic Regression, as well as non-parametric and more computationally intensive ones, such as Random Forest or Stochastic Gradient Descent. The classifiers were imported in Python language from the Sklearn library and trained using the parameters' settings detailed in Appendix A.

2.3.4. Step 4: Evaluation

The comparison of word embeddings and classifiers focused on analysis of time consumption and classifiers' performance and included various processing set-ups.

To assess one of the practical aspects of the methods considered, the time taken to extract the features using WEs and time to train the classifier was measured. To measure the time taken in parts of the training, the time function from Python was used. Each time measurement was repeated 5 times and the average reported to decrease any possible bias.

Moreover, to evaluate the classifiers' performance, precision, recall and F1-Score (Powers, 2011) were computed. Although the F1-Score is a less intuitive metric than accuracy, it is a popular choice in the literature as it better reflects incorrectly classified instances by taking into account how the data are distributed. The evaluation measures used in this experiment are macro averages calculated as the unweighted mean of precision, recall and F1-Score for all classes in the outcome variable given by the formulae below:

$$\text{Macro-Average-Precision} = \frac{1}{K}\sum_{i=1}^{K}\text{Precision}_i = \frac{1}{K}\sum_{i=1}^{K}\frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i},$$

$$\text{Macro-Average-Recall} = \frac{1}{K}\sum_{i=1}^{K}\text{Recall}_i = \frac{1}{K}\sum_{i=1}^{K}\frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i},$$

$$\text{Macro-Average-F1-Score} = \frac{1}{K}\sum_{i=1}^{K}\text{F1-Score}_i = \frac{1}{K}\sum_{i=1}^{K}\frac{2\cdot\text{Precision}_i\cdot\text{Recall}_i}{\text{Precision}_i + \text{Recall}_i},$$

where K is the number of classes, TP_i is the number of True Positives for class i, FP_i is the number of False Positives for class i, FN_i is the number of False Negatives for class i. The above evaluation measures were calculated using five-fold cross-validation and the *classification_report* method from the Sklearn library in Python was employed.

In this study, we used the Google Colab environment[2] to run the Python code for all processes, which is a cloud platform widely used in machine learning and deep learning applications. See Carneiro et al. (2018) for an early discussion of it. The use of this environment allowed us to study and compare the performance of the algorithms on the CPU and GPUs. In particular, for the BERT word embedding algorithm, in most cases the code would not run with the standard memory options for the CPUs and GPUs and so the HighRAM option needed to be employed. CPU-HighRAM and T4-GPU-HighRAM processing sets were tested, by measuring the feature extraction time for each word embedding and the training time for each classifier. Furthermore, when studying the energy consumption and CO2 emissions of word embeddings in Section 3.5, seven processing sets were considered: T4-GPU-HighRAM, CPU-HighRAM, CPU, T4-GPU, A100-GPU-HighRAM, V100-GPU, V100-GPU-HighRAM.

3. Results

In this section, the results of experiments are presented and analyzed. Section 3.1 includes a comparison between word embeddings with respect to the time consumed by the feature extraction algorithms. Two processing sets are considered CPU-HighRAM and T4-GPU-HighRAM. Section 3.2 is focused on classifiers' training times and comparisons between classifiers and between word embeddings are made. In Section 3.3, the results on classifiers' performance metrics are analyzed. This includes comparisons between word embeddings (Section 3.3.1), comparisons between the Average and PCA approaches to feature extraction (Section 3.3.2) and the overall comparison between classifiers (Section 3.3.3). In Section 3.4 the patterns for the relationships between the consumed time and the classifiers' performance are discussed. Finally, the energy consumed and the CO2 emissions produced are discussed in Section 3.5.

3.1. Feature extraction times

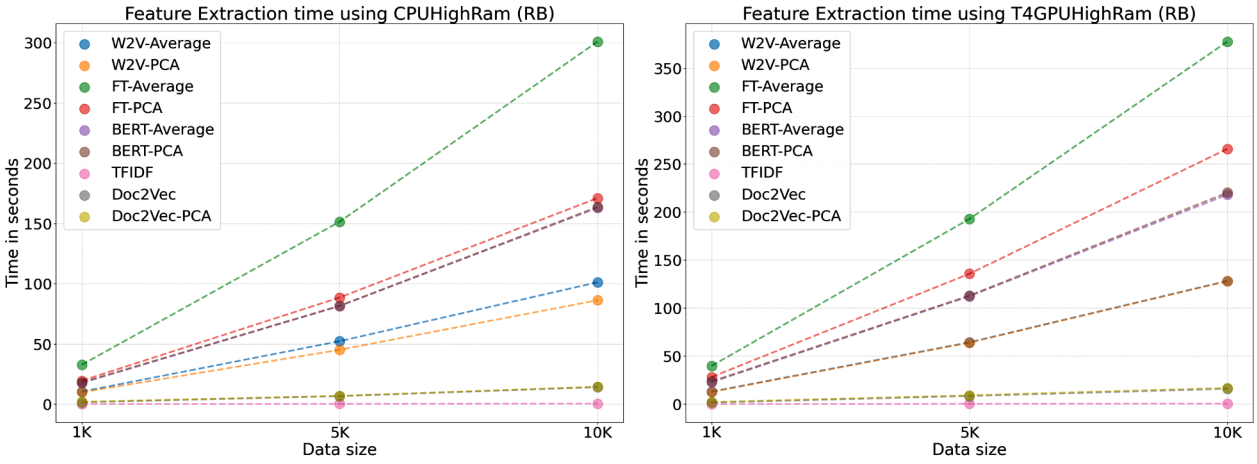
In this subsection, word embeddings are compared with respect to the time consumed in the empirical study. Two processing sets are considered: CPU-HighRAM and T4-GPU-HighRAM. The results for the feature extraction times for the nine types of text representation and two processing settings are visualized in Fig. 2.

Unsurprisingly, the TF-IDF method needed the least amount of time. Doc2Vec was consistently the second fastest word embedding. Both, Word2Vec and BERT required considerably longer time than Doc2Vec and their relative speeds varied depending on the processor and dataset, with Word2Vec being faster than BERT in all cases except two. FastText word embeddings needed the longest time in every scenario and usually required roughly 400 times more time than TF-IDF and 20 times more than Doc2Vec. Similar patterns held for all three datasets, processor and memory settings and sample sizes: 1000, 5000 and or 10,000.

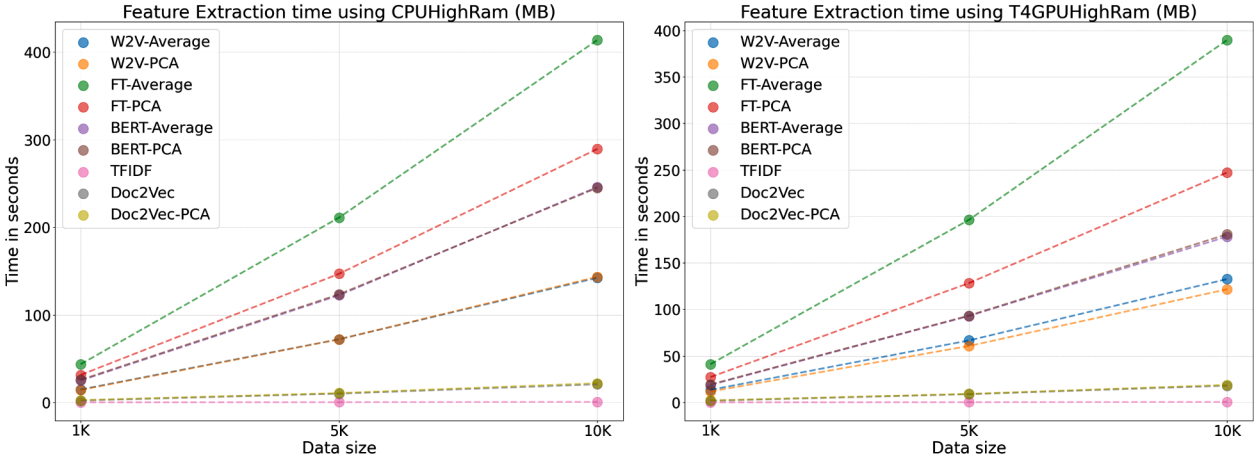
Studying the effect of increasing the sample size on the feature extraction time, as visualized in Fig. 2, TF-IDF and Doc2Vec had a relatively constant feature extraction time for the three dataset sizes, which means that increasing the data size five and ten times had a negligible effect on the needed time. On the other hand, FastText's time increased with the sample size at the steepest rate among all other word embedding types, followed by BERT and Word2Vec. The increase rates were approximately linear in each case.

When comparing the results for the two processing powers, in most cases CPU-HighRAM needed slightly more time than T4-GPU-HighRAM and the increase typically ranged from 1 % to 10 %. However, for the Radical-Binary dataset, the opposite effect occurred, and CPU-HigRAM was slightly faster than T4-GPU-HighRAM. Generally, the processing power appeared to play an important role for BERT, where the biggest

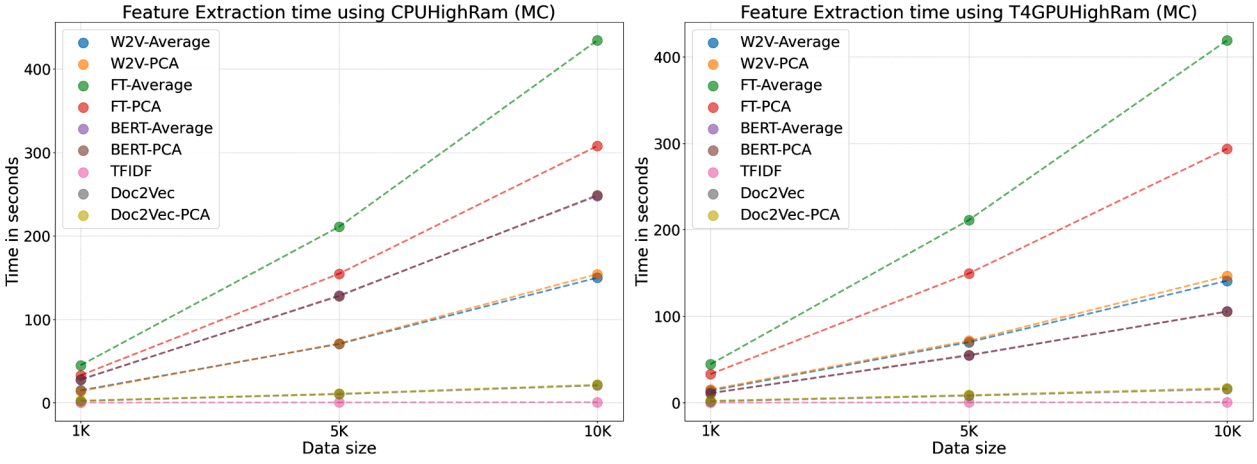
[2] https://colab.research.google.com/



(a) Radical Binary data



(b) Mixed Binary data



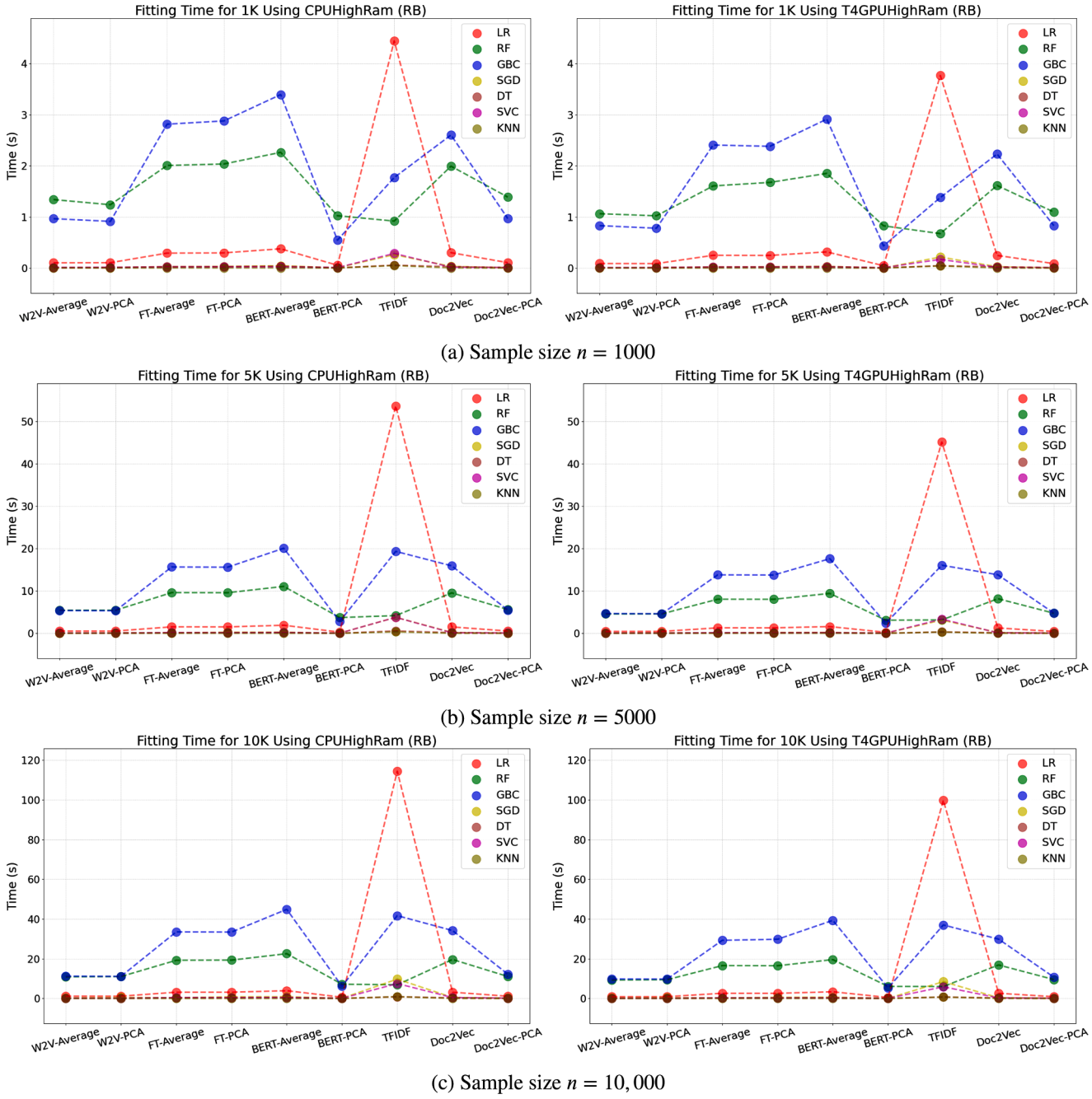
(c) Multi-Class data

Fig. 2. Feature extraction time versus sample size for 1K, 5K and 10K sample sizes, processed by CPU-HighRAM (left panel) and T4-GPU-HighRam (right panel) for (a) Radical Binary data, (b) Mixed Binary data and (c) Multi-Class data.

differences can be observed as in some cases CPU-HighRAM needed almost double the time than T4-GPU-HighRAM for BERT embeddings.

3.2. Classifiers' training times

In this subsection, the training times of classifiers are analyzed. Again, two processing sets are considered: CPU-HighRAM and T4-GPU-HighRAM. The observed classifiers' times for the nine scenarios of text representations and two processing settings are visualized in Figs. 3–5 for Mixed Binary, Radical Binary and Multi-Class dataset, respectively. The graphs show curves of the fitting times against WEs and each curve corresponds to one classifier, with a separate graph for each sample size, for an easy comparison between classifiers and WEs when the sample size is fixed. Moreover, Fig. 6 visualizes the effect of the sample size on



(a) Sample size $n = 1000$

(b) Sample size $n = 5000$

(c) Sample size $n = 10,000$

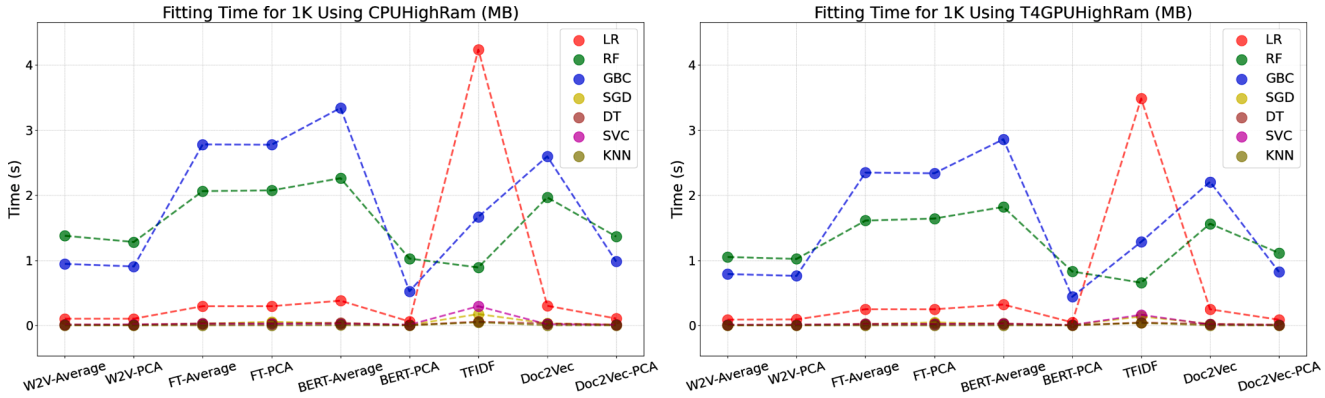
Fig. 3. Fitting time for classifiers for 1K, 5K and 10K sample sizes using CPU-HighRAM (left-hand side panel) and T4-GPU-HighRAM (right-hand side panel) for nine types of word embedding as indicated on the horizontal axes, for Radical Binary data.

fitting times for each WE with a separate graph for each classifier, for the Radical Binary dataset and CPU-HighRAM processing setting as an example since similar trends were observed for all three datasets.

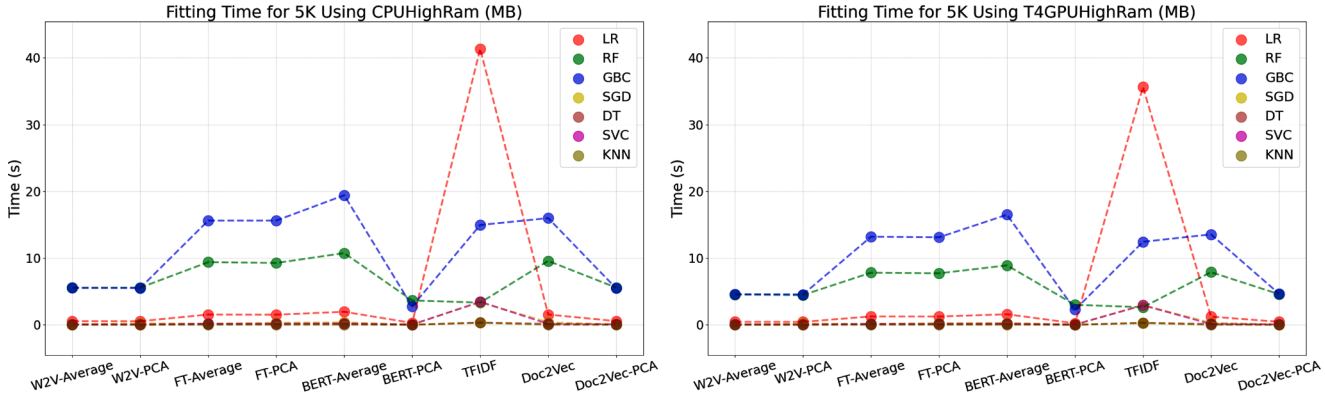
For all considered classifiers except the two ensemble methods, Gradient Boosting and Random Forest, using TF-IDF text representation resulted in the slowest training process. This is illustrated in Fig. 6 by the pink curves being above all other curves in graphs (a), (c), (d), (f) and (g). In particular, for the Radical Binary and Mixed Binary datasets, TF-IDF combined with logistic regression stood out as the slowest among all other WEs and classifiers' combinations, which can be observed as the distinctive peaks on the red curves in Figs. 3 and 4.

However, for Gradient Boosting and Random Forest, using BERT-Average required more time than any other WE, including TF-IDF. As expected, these two ensemble classifiers, marked by the blue and green curves in Figs. 3–5, generally required more time than the other methods for all WEs, which is in line with general properties of these algorithms. For Gradient Boosting, when using BERT-PCA, Doc2Vec-PCA and the two Word2Vec text representations the process of training was at least twice as fast as when using BERT-Average, TF-IDF, Doc2Vec and the two FastText WEs. The same conclusions can be made for Random Forest with one difference appearing for TF-IDF which seemed to work relatively fast for this classifier.

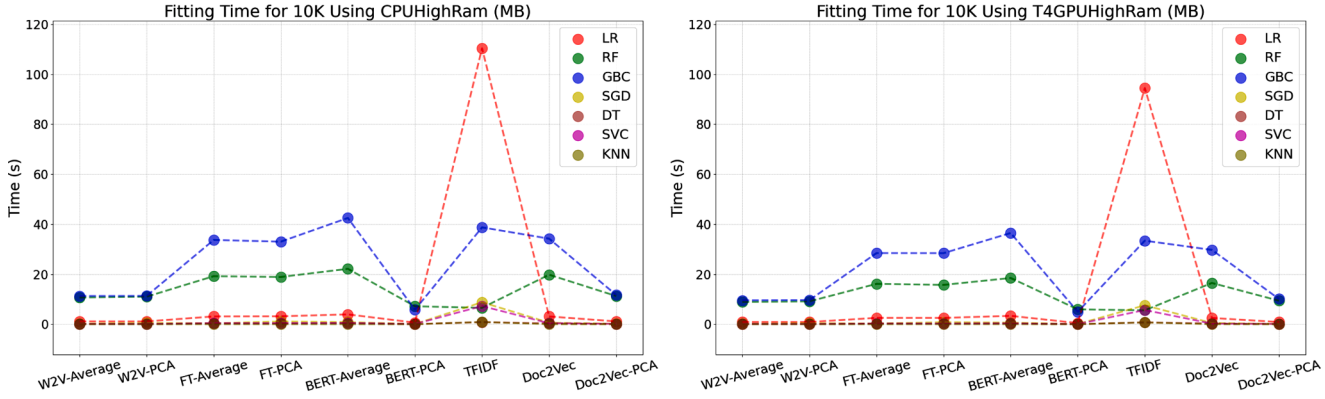
When increasing the sample size from 1000 to 5000 and 10,000, the training time grew in an approximately linear manner for all WEs and the steepest increase can be noticed for TF-IDF in most cases. For most classifiers, the increase was the least steep for BERT-PCA, D2V-PCA, W2V-Average and W2V-PCA. On the other hand, it was generally much steeper for BERT-Average, FT-Average, FT-PCA and Doc2Vec.



(a) Sample size $n = 1000$



(b) Sample size $n = 5000$



(c) Sample size $n = 10,000$

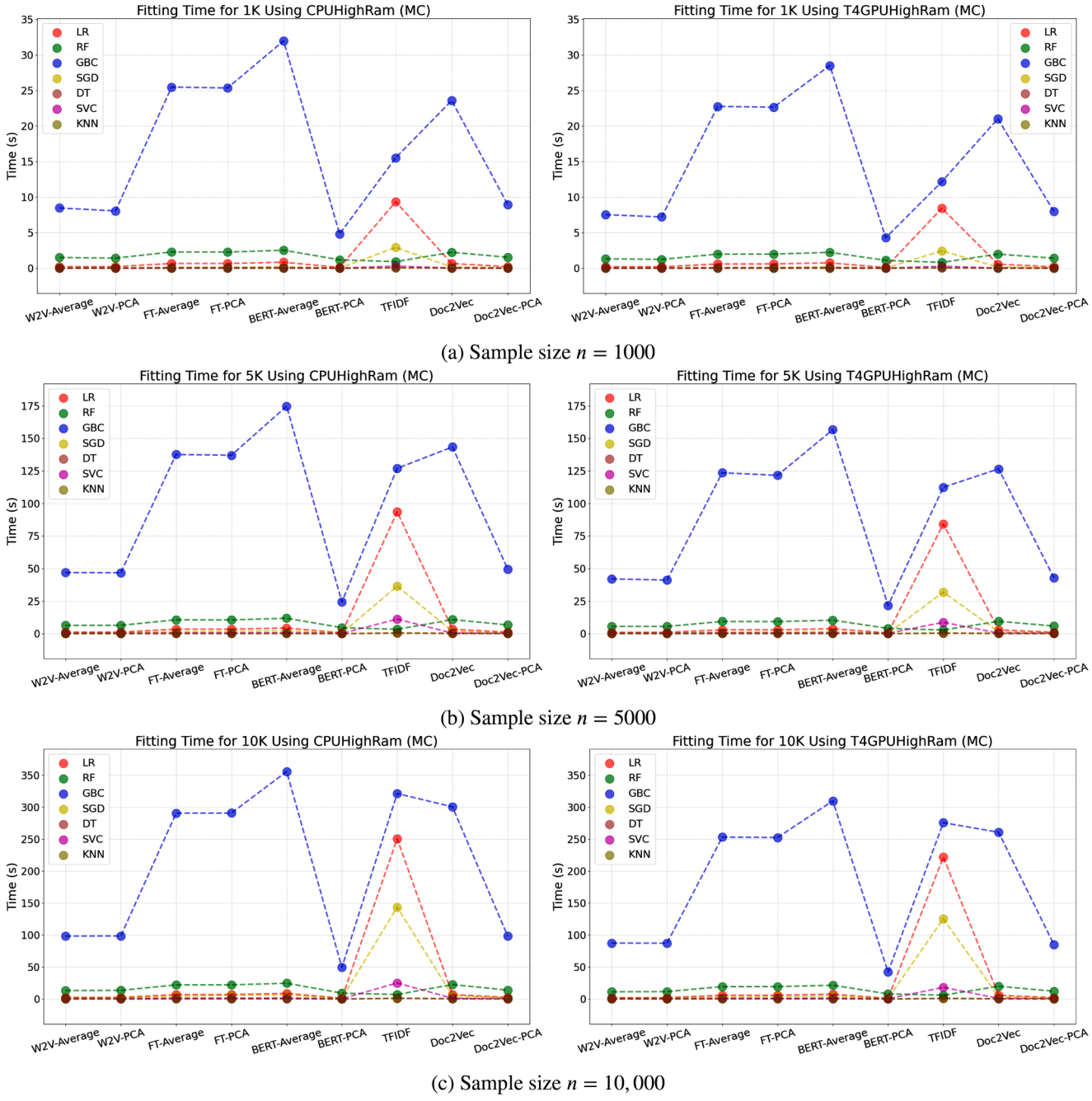
Fig. 4. Fitting time for classifiers for 1K, 5K and 10K sample sizes using CPU-HighRAM (left-hand side panel) and T4-GPU-HighRAM (right-hand side panel) for nine types of word embedding as indicated on the horizontal axes, for Mixed Binary data.

Overall, the same patterns held for both processing settings: CPU-HighRAM and T4-GPU-HighRAM, with the latter being marginally faster, which can be observed by comparing the graphs on the left- and right-hand side in Figs. 3–5.

In summary, BERT-PCA stood out as a text representation method that yielded relatively fast fitting time for all classifiers and was closely followed by Doc2Vec-PCA, Word2Vec-Average and Word2Vec-PCA. For BERT, applying PCA as opposed to taking the average of features (BERT-Average) yielded faster training of classifiers. Similarly, Doc2Vec-PCA worked faster than Doc2Vec for most classifiers. However, the same effect was not observed for FastText and Word2Vec. That is, FT-Average and FT-PCA required a similar amount of time for classifiers' training, as was the case for W2V-Average and W2V-PCA. Moreover, using the two Word2Vec-based WEs generally led to faster training of classifiers than the two FastText-based features.

3.3. Classifiers' performance

In this section, we focus on the performance of each classifier in terms of macro-averages for F1-Score, precision and recall and assess the effects of the type of WEs used in the process of training. Moreover, two approaches to feature extraction from WEs are compared: taking averages of word vectors and applying a PCA-based approach. Finally, the classifiers are compared with respect to their overall performance. The results are visualized in Figs. 8–10 showing macro-average curves of F1-Scores against WEs and each curve corresponds to one classifier,



(a) Sample size $n = 1000$

(b) Sample size $n = 5000$

(c) Sample size $n = 10,000$

Fig. 5. Fitting time for classifiers for 1K, 5K and 10K sampel sizes using CPU-HighRAM (left-hand side panel) and T4-GPU-HighRAM (right-hand side panel) for nine types of word embedding as indicated on the horizontal axes, for Multi-Class data.

with a separate graph for each sample size. For precision and recall, very similar patterns were observed as those visible in the graphs for F1-Scores.

3.3.1. *Comparison between word embeddings*

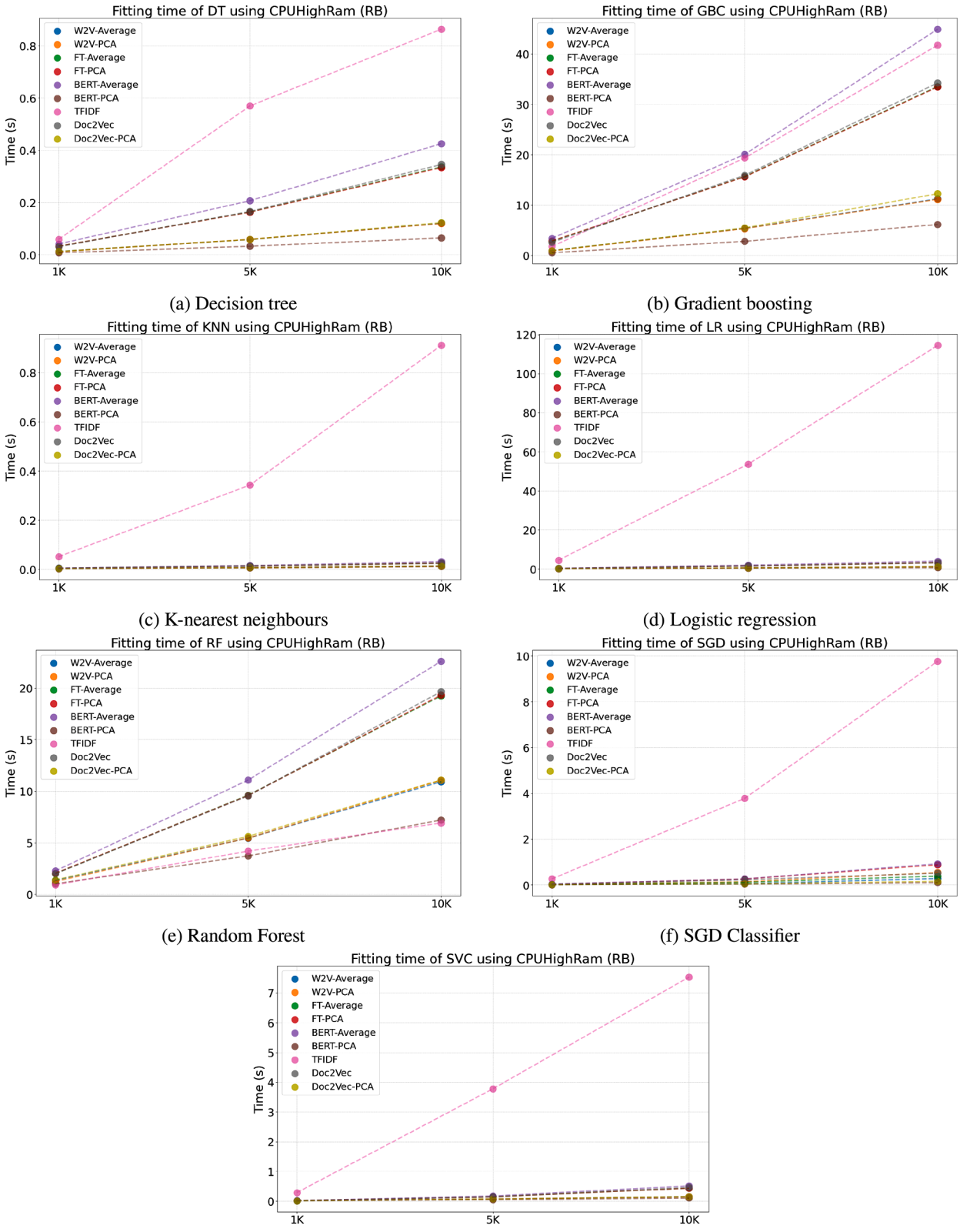
In this subsection, the performance of WEs is compared for each classifier separately.

Comparison of word embeddings for Random Forests

For Random Forests (RFs), using TF-IDF consistently led to the lowest F1-Scores, precisions and recalls for all datasets and all sample sizes. This can be observed as a distinct dip in the yellow curves in Figs. 8–10, where RFs applied to TF-IDF achieved F1-Scores ranging between 0.86 and 0.9 for Radical Binary dataset, between 0.85 and 0.87 for Mixed Binary dataset and between 0.53 and 0.59 for the Multi-Class dataset, respectively.

For Radical Binary and Mixed Binary datasets, almost all of the other WEs led to very similar results and achieved an excellent performance of RFs with F1-Scores in the narrow range between 0.95 and 0.97 for: Word2Vec-Aver, Word2Vec-PCA, FastText-Aver, BERT-Aver, BERT-PCA and Doc2Vec-PCA. For FastText-PCA and Doc2Vec only slightly lower scores were observed: between 0.91 and 0.93 for FastText-PCA and between 0.89 and 0.94 for Doc2Vec.

However, for the most challenging Multi-Class datasets, the performance of WEs was more varied and BERT-PCA stood out with the highest F1-Scores for RFs equal to 0.9 for all three sample sizes. This is in contrast with the lowest F1-Scores ranging between 0.53 and



(a) Decision tree

(b) Gradient boosting

(c) K-nearest neighbours

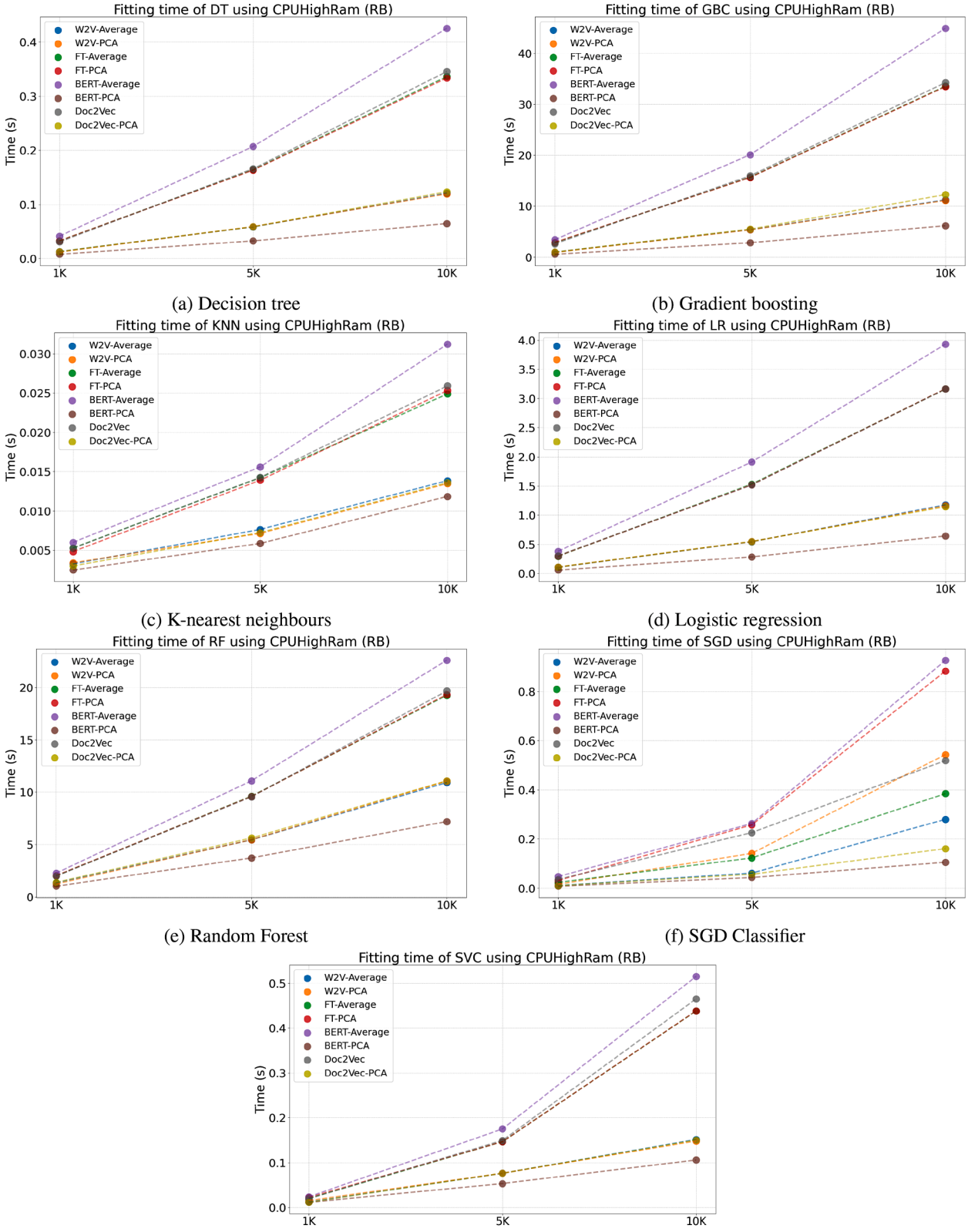
(d) Logistic regression

(e) Random Forest

(f) SGD Classifier

(g) Support Vector Machine Classifier

Fig. 6. Classifiers' training times for 1K, 5K and 10K sample sizes using CPU-HighRAM for Radical Binary data. Each graph shows classifier training times versus dataset size, with a separate curve for each word embedding. Please see Fig. 7 for a clearer visualisation of the same, obtained by excluding TF-IDF results.



(a) Decision tree

(b) Gradient boosting

(c) K-nearest neighbours

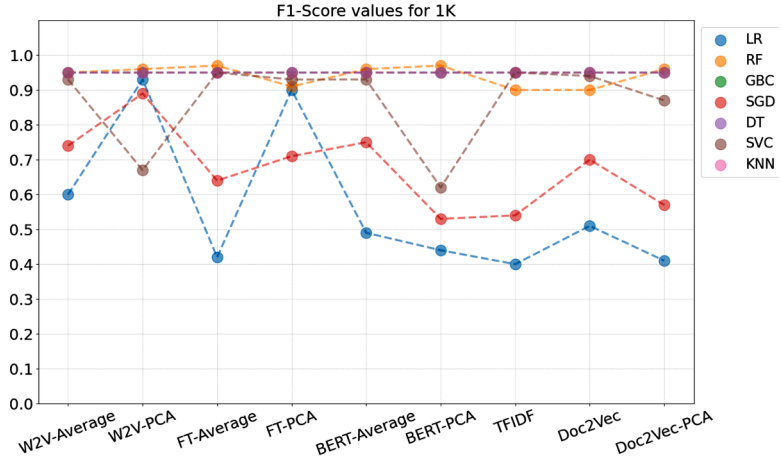
(d) Logistic regression

(e) Random Forest

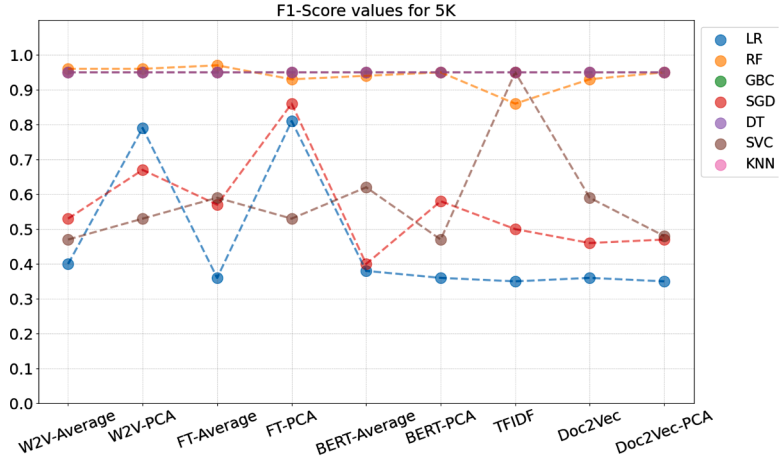
(f) SGD Classifier

(g) Support Vector Machine Classifier

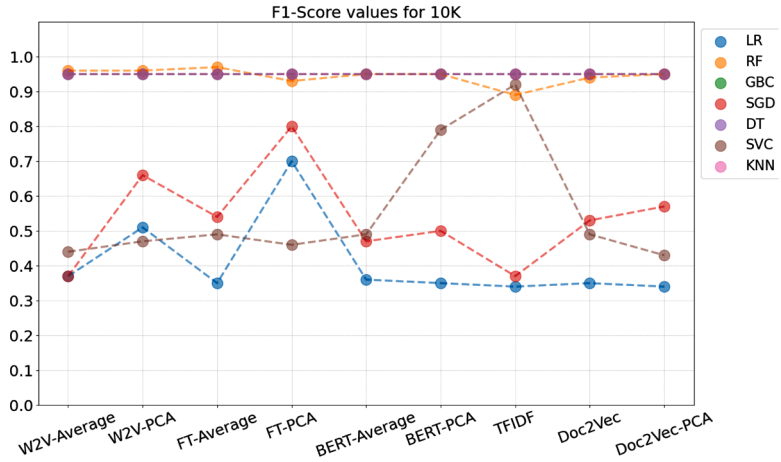
Fig. 7. Classifiers' training times for 1K, 5K and 10K sample sizes using CPU-HighRAM for Radical Binary data. Each graph shows classifier training times versus dataset size, with a separate curve for each word embedding, where TF-IDF is excluded for clarity of the presentation.



(a) $n = 1000$

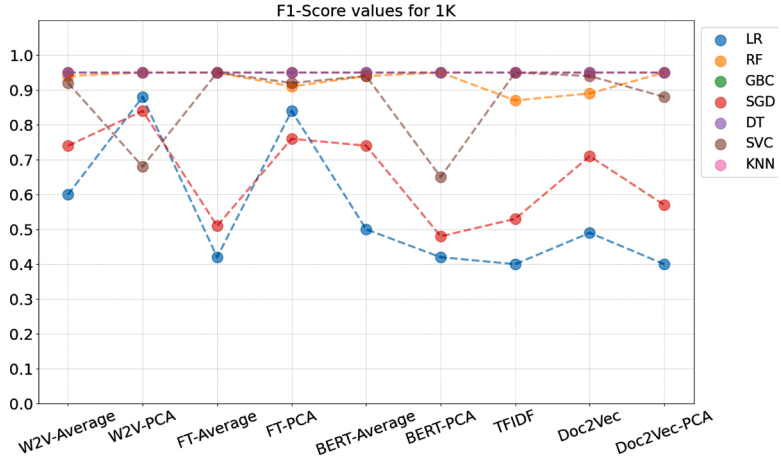


(b) $n = 5000$

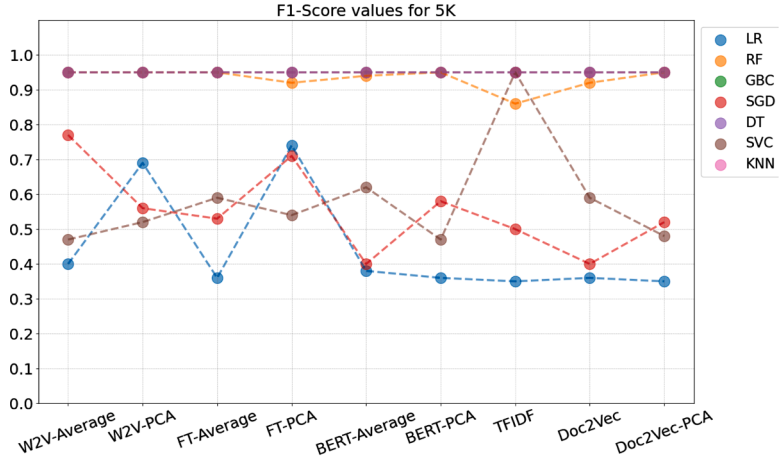


(c) $n = 10,000$

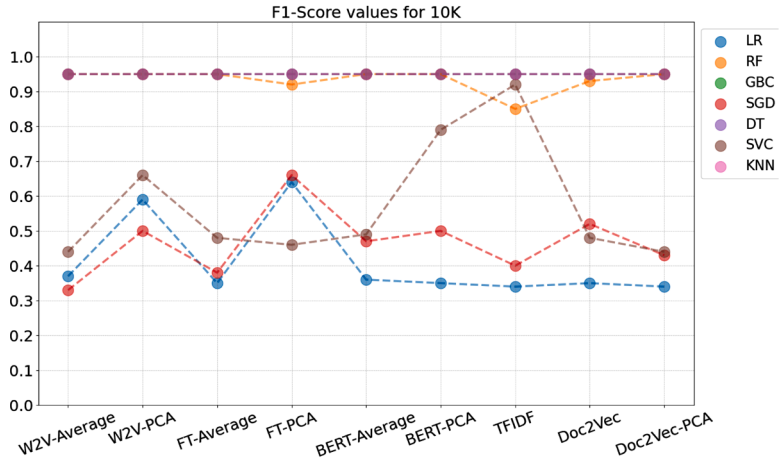
Fig. 8. F1-Scores for seven classifiers, each marked with a different colour, and nine WEs (marked on horizontal axis) for Radical Binary dataset with sample size of (a) $n = 1000$, (b) $n = 5000$ and (c) $n = 10,000$. In all three graphs, the horizontal purple-pink line at the top represents GBC, DT and KNN as they have the same F1-Scores equal 0.95 for all WEs.



(a) $n = 1000$

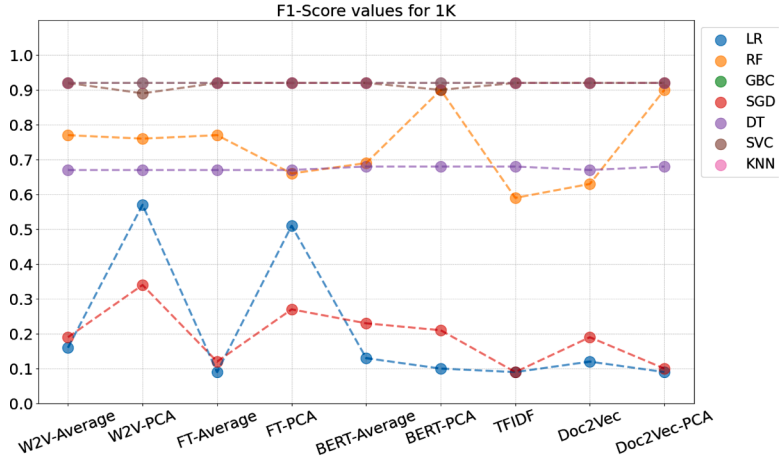


(b) $n = 5000$

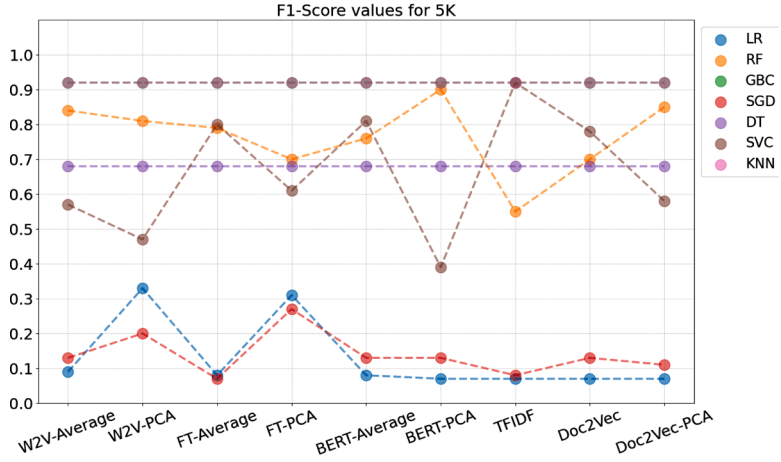


(c) $n = 10,000$

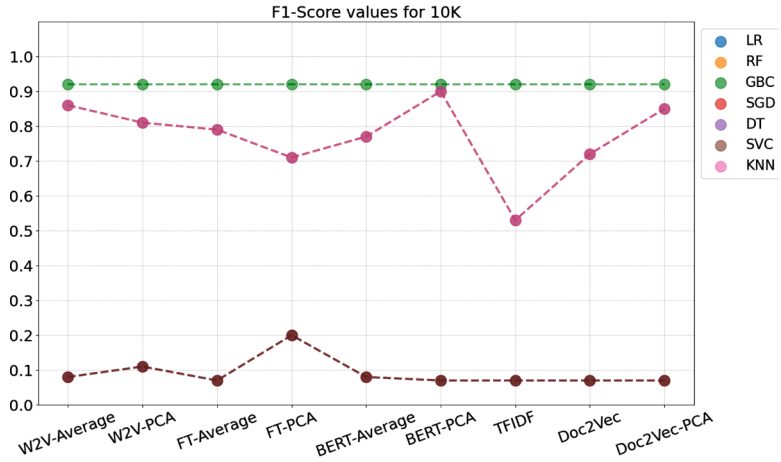
Fig. 9. F1-Scores for seven classifiers, each marked with a different colour, and nine WEs (marked on horizontal axis) for Mixed Binary dataset with sample size of (a) $n = 1000$, (b) $n = 5,000$ and (c) $n = 10,000$. In all three graphs, the horizontal purple-pink line at the top represents GBC, DT and KNN as they have the same F1-Scores equal 0.95 for all WEs.



(a) $n = 1000$

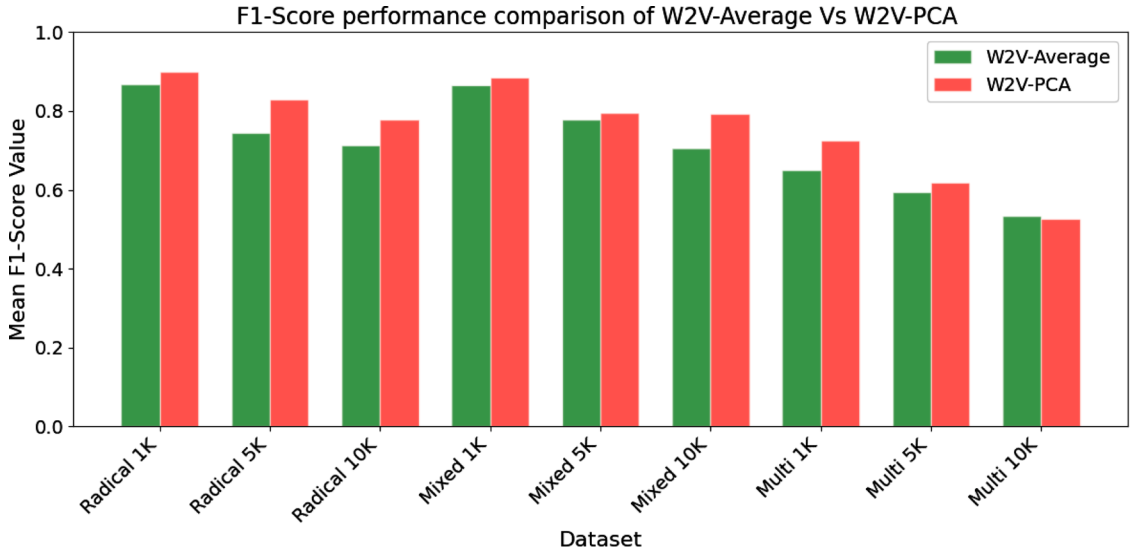


(b) $n = 5000$

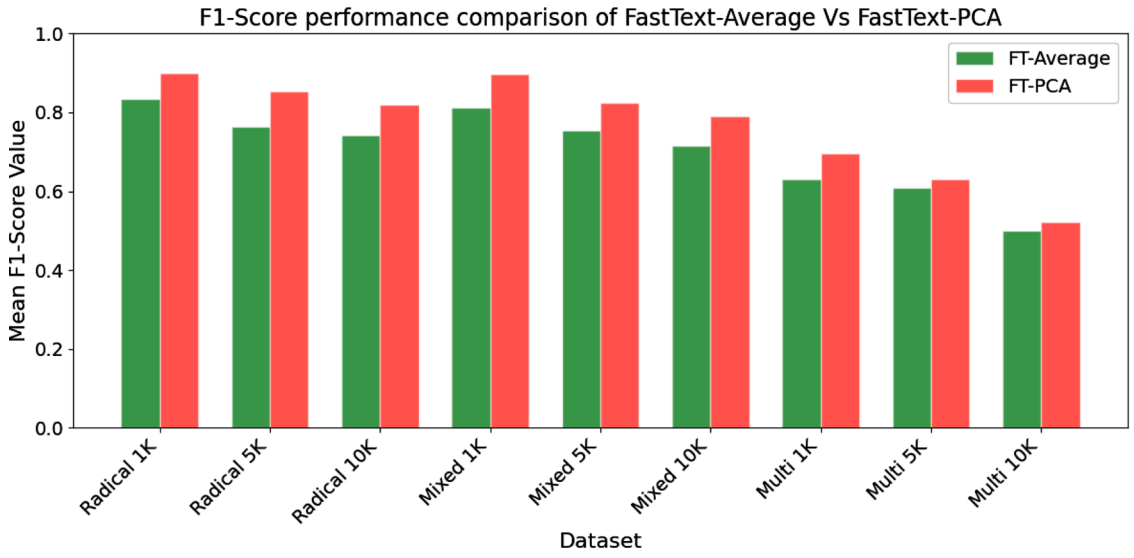


(c) $n = 10,000$

Fig. 10. F1-Scores for seven classifiers, each marked with a different colour, and nine WEs (marked on horizontal axis) for Multi-Class dataset with sample size of (a) $n = 1000$, (b) $n = 5000$ and (c) $n = 10,000$. In graphs (a) and (b), the horizontal line at the top represents both GBC and KNN as they have the same F1-Scores equal 0.92 for all WEs. In graph (c), the bottom curve represents LR, RF, SGD, DT and SVC as they have the same F1-Scores.



(a) Comparison of Word2Vec-Average vs Word2Vec-PCA



(b) Comparison of FastText-Average vs FastText-PCA

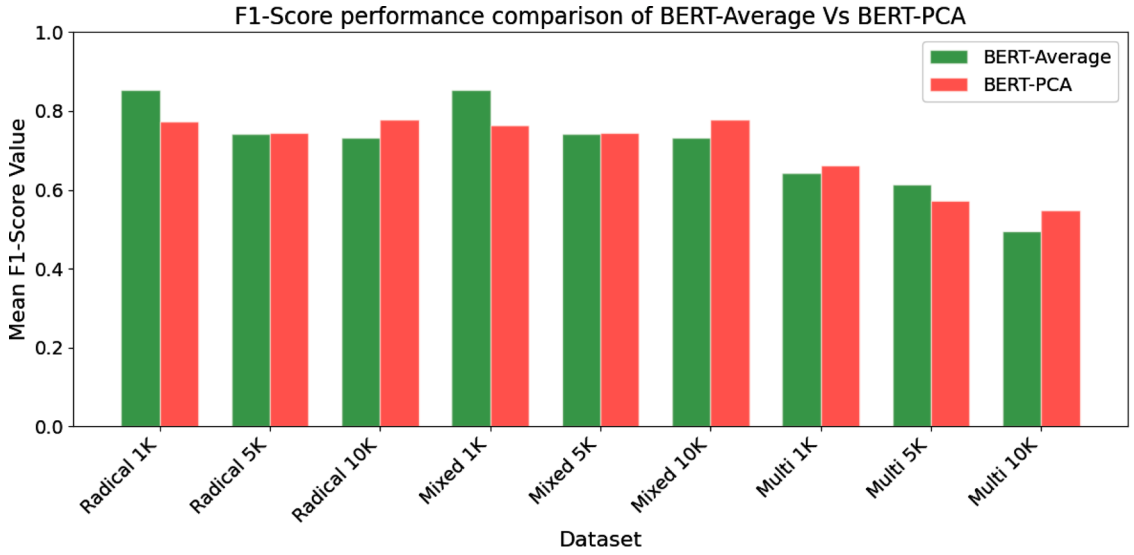
Fig. 11. The averages of F1-Scores over seven classifiers for each dataset (marked on the horizontal axis), for Word2Vec-Average vs Word2Vec-PCA (top graph) and FastText-Average vs FastText-PCA (bottom graph).

0.59 when using TF-IDF for the Multi-Class datasets. Doc2Vec-PCA performed well for this dataset with F1-Scores between 0.85 and 0.9 and Word2Vec-Aver performed very similarly to Doc2Vec-PCA for $n = 5,000$ (F1-Score = 0.84) and $n = 10,000$ (F1-Score = 0.86) but not as good for the smallest dataset (F1-Score = 0.77 for $n = 1,000$). On the other hand, Doc2Vec was the second worst after TF-IDF, with F1-Scores between 0.63 and 0.72 and FastText-PCA gave very similar results to Doc2Vec (F1-Scores ranging between 0.66 and 0.71). BERT-Average is marginally better than Doc2Vec with F1-Scores ranging between 0.69 for $n = 1,000$ and 0.77 for $n = 10,000$. Finally, FastText-Aver and Word2Vec-PCA performed similarly to each other with F1-Scores between 0.77 and 0.81, which placed them above TF-IDF, Doc2Vec, FastText-PCA and BERT-Aver but below Word2Vec-Aver, Doc2Vec-PCA and BERT-PCA, ordered from the worst to best-performing model.

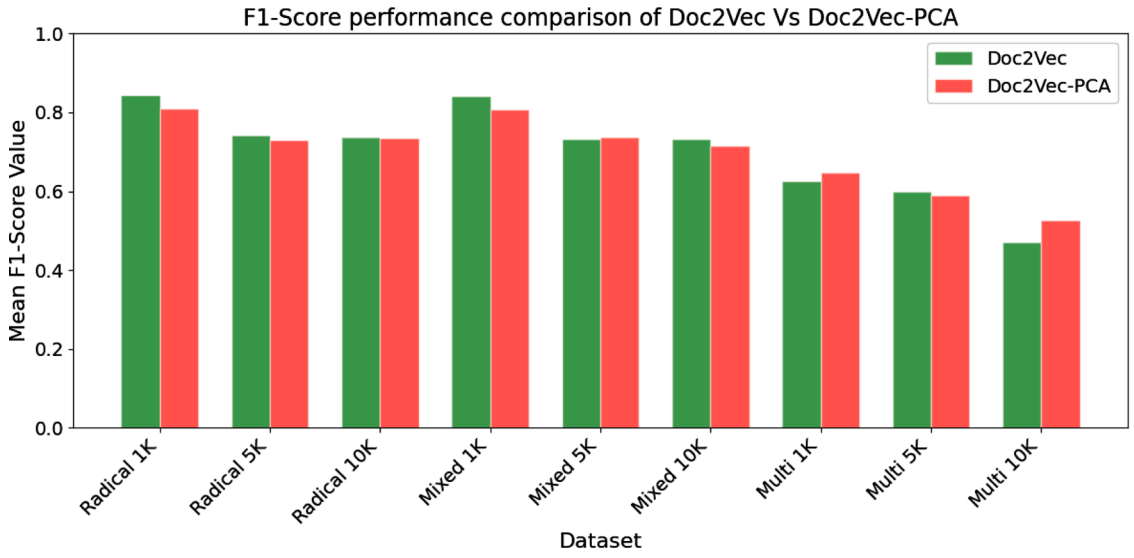
In summary, the WEs that stood out for RFs are: TF-IDF, which consistently led to the lowest scores for all datasets and BERT-PCA, which achieved the highest scores for the most challenging classification tasks, closely followed by Doc2Vec-PCA with very similar scores.

Comparison of word embeddings for Logistic Regression

For Logistic Regression (LR), Word2Vec-PCA and FastText-PCA provided consistently better precisions, recalls and F1-Scores over all other WEs. This held true for all three datasets and sample sizes. For example, for the Radical Binary dataset of size $n = 1000$, F1-Scores of 0.93 and 0.9 were observed for Word2Vec-PCA and FastText-PCA, respectively, while all other WEs ranged between 0.4 (TF-IDF) and 0.6 (Word2Vec-Aver). For another example, for the Multi-Class dataset of size $n = 1000$, F1-Scores of 0.57 and 0.51 were observed for Word2Vec-PCA and



(a) Comparison of BERT-Average vs BERT-PCA



(b) Comparison of Doc2Vec vs Doc2Vec-PCA

Fig. 12. The averages of F1-Scores over seven classifiers for each dataset (marked on the horizontal axis), for BERT-Average vs BERT-PCA (top graph) and Doc2Vec vs Doc2Vec-PCA.

FastText-PCA, respectively, while all other WEs ranged between 0.09 (TF-IDF) and 0.16 (Word2Vec-Aver) and the advantage of using Word2Vec-PCA or FastText-PCA is more pronounced for this most challenging dataset. The same pattern held for the datasets of sizes 5000 and 10000, however, the scores became lower as the sample size increased which suggests possible over-fitting of the model.

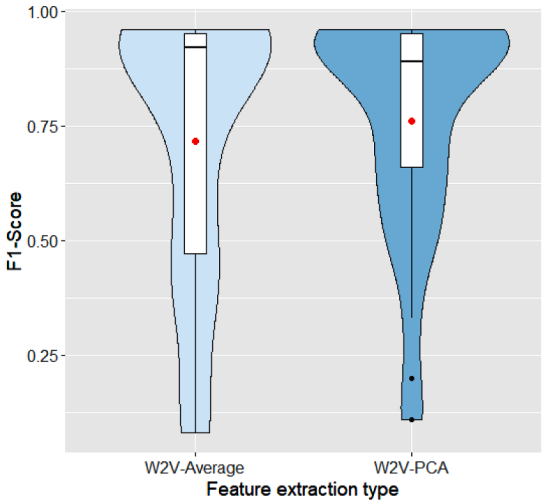
The TF-IDF method performed generally very poorly for LR, however its scores were very close to several other WEs with similar poor results. In particular, in most cases, Doc2Vec-PCA achieved the same F1-Scores as TF-IDF ranging between 0.35 and 0.4 for the Radical Binary and Mixed Binary datasets and between 0.07 and 0.09 for the Multi-Class datasets.

Comparison of word embeddings for Stochastic Gradient Decent

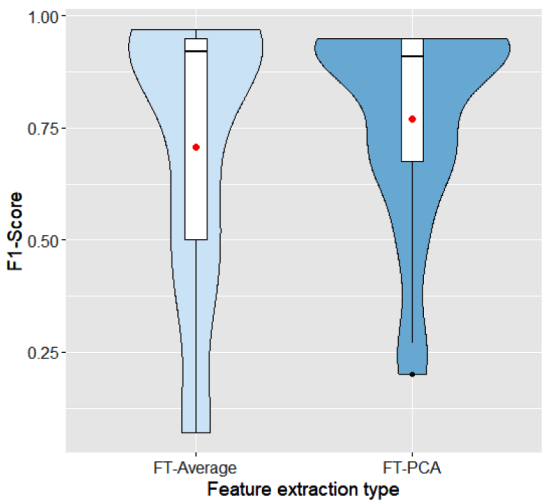
For Stochastic Gradient Decent (SGD), the patterns were similar to those observed for LR in that Word2Vec-PCA and FasText-PCA led to the best scores in most cases. However, there are also a few differences here, in particular for the BERT WEs which performed relatively well in some cases and poorly in others.

For example, for the Radical Binary dataset of size $n = 1000$, the best F1-Scores of 0.89, 0.75 and 0.74 were observed for Word2Vec-PCA, BERT-Average and Word2Vec-Average, respectively, while all other WEs ranged between 0.53 (BERT-PCA) and 0.71 (FastText-PCA). For another example, for the Mixed Binary dataset of size $n = 1000$, the highest F1-Scores of 0.84 and 0.76 were observed for Word2Vec-PCA and FastText-PCA, respectively, while all other WEs ranged between 0.48 (BERT-PCA) and 0.74 (W2V-Average and BERT-Average). Finally, for the Multi-Class dataset of size $n = 1000$, the highest F1-Scores of 0.34 and 0.27 were observed for Word2Vec-PCA and FastText-PCA, respectively, while all other WEs ranged between 0.09 (TF-IDF) and 0.23 (BERT-Average).

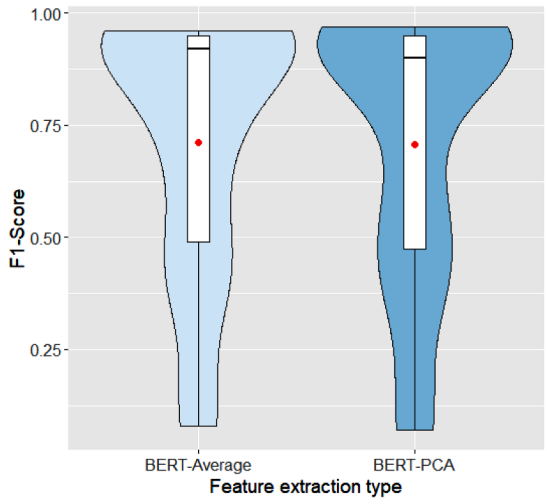
In summary, for the SGD classifier, Word2Vec-PCA and FasText-PCA stood out as leading to the best scores in most cases. However, there is



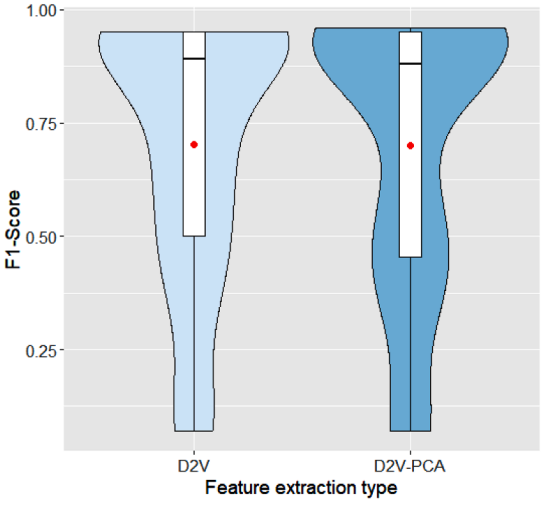
(a) Comparison between Word2Vec-Average vs Word2Vec-PCA



(b) Comparison between FastText-Average vs FastText-PCA



(c) Comparison between BERT-Average vs BERT-PCA



(d) Comparison between Doc2Vec vs Doc2Vec-PCA

Fig. 13. Violin plots showing the distribution of 63 F1-Score values, over nine datasets and seven classifiers, for Word2Vec-Average vs Word2Vec-PCA (graph (a)), FastText-Average vs FastText-PCA (graph (b)), BERT-Average vs BERT-PCA (graph (c)) and Doc2Vec vs Doc2Vec-PCA (graph (d)). The red points indicate the mean value.

no clear pattern regarding which WE leads to the lowest scores here, with TF-IDF, BERT-PCA and BERT-Average being the worst on some occasions, but not consistently.

Comparison of word embeddings for Support Vector Classifier

For SVC, a different pattern occurred in comparison to other classifiers. Surprisingly, in most cases, TF-IDF text representation yielded the highest F1-Scores, precisions and recalls. This can be seen as a distinct peak in the brown curves in Figs. 8 (b and (c), 9 (b) and (c) and 10 (b), where the F1-Scores for SVC with TF-IDF text embedding were between 0.92 and 0.95, roughly twice as high as for the lowest scoring WEs.

For the Radical Binary dataset of size $n = 1000$, F1-Scores of 0.95 for both FastText-Average and TF-IDF and 0.94 for Doc2Vec-Average were observed, while all other WEs ranged between 0.62 (BERT-PCA) and 0.93 (W2V-Average). For the Radical Binary dataset of size $n = 5000$, the F1-Score for TF-IDF was still high (0.95) but all the other WEs yielded lower F1-Scores ranging between 0.47 for Word2Vec-Average and BERT-PCA and 0.62 for BERT-Average. For the Mixed Binary dataset of size $n = 1000$, F1-Scores of 0.95 for both FastText-Average and TF-IDF and 0.94 for Doc2Vec-Average and BERT-Average were observed, while all other WEs ranged between 0.65 (BERT-PCA) and 0.92 (W2V-Average). For the Multi-Class dataset of size $n = 1000$, F1-Scores of 0.92 for seven classifiers Word2Vec-Average, FastText-Average, FastText-PCA, BERT-Average, TF-IDF, Doc2Vec-Average and Doc2Vec-PCA were observed, while Word2Vec-PCA was 0.89 and BERT-PCA was 0.90.

Generally, BERT-PCA stood out for the SVC classifier as yielding the lowest F1-Scores in most cases for sample sizes 1000 and 5000, while TF-IDF stood out as the best-performing WE. Also, in many cases, the SVC classifier achieved better scores for smaller sample sizes, which can be indicative of overfitting.

Comparison of word embeddings for Decision Tree

For Decision Tree (DT), it was found that in almost all cases all WE algorithms had the same performance. For the Radical Binary dataset, F1-Scores and recalls were always 0.95 and precisions were always 0.97, for all sample sizes and all WEs. The same pattern occurred for the Mixed Binary dataset.

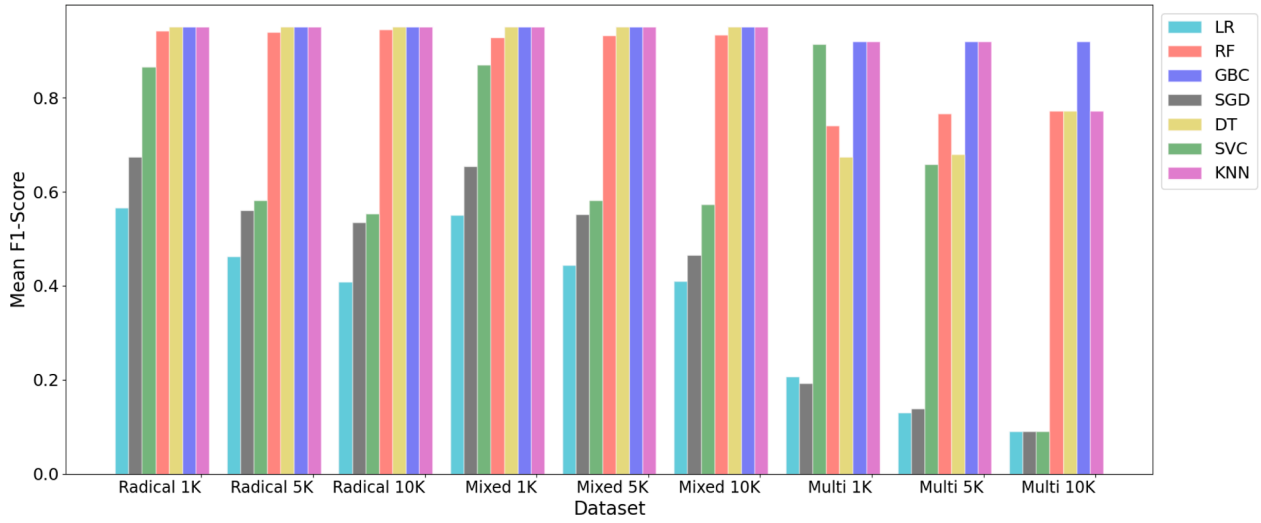


Fig. 14. Performance comparison between classifiers showing the average of F1-Score values over nine WEs, for each classifier and each dataset, where "Radical 1K" denotes the Radical Binary dataset of size 1,000, "Mixed 1K" denotes the Mixed Binary dataset of size 1,000, "Multi 1K" denotes the Multi-Class dataset of size 1,000, etc.

For the Multi-Class dataset of size $n = 1000$, the F1-Scores varied only minimally, but this time they were lower and ranging between 0.67 and 0.68, while for $n = 5000$, all WEs again yielded the same F1-Scores, equal to 0.67. Finally, for $n = 10,000$, different performances of WEs were observed with the highest F1-Score (0.9) achieved by BERT-PCA and the lowest (0.53) for TF-IDF.

In summary, for DTs, any differences between WEs occurred only for the Multi-Class dataset of size $n = 10,000$. Where differences occurred, BERT-PCA yielded the highest F1-Score and TF-IDF the lowest. For the remaining eight datasets, the DT classifier was not sensitive to the type of WE employed.

Comparison of word embeddings for KNN

Following the same pattern as DTs, the KNN classifiers had identical values of F1-Scores (0.95), precisions (0.97) and recalls (0.95) for all WEs for the Radical and Mixed Binary dataset and for all sample sizes. For the Multi-Class datasets of size $n = 1000$ and $n = 5000$, the KNN classifiers also had identical F1-Scores of 0.92 for all WEs, while for size $n = 10,000$ a variety of values for F1-Score were observed, with a minimum of 0.53 for TF-IDF and a maximum of 0.9 for BERT-PCA.

Comparison of word embeddings for Gradient Boosting

For Gradient Boosting Classifiers (GBCs), a very similar pattern as for DTs and KNN method was observed. The GBCs had the same value of F1-Score (0.95), precision (0.97) and recall (0.95) for all WEs for both Radical Binary and Mixed Binary datasets of all three sample sizes. For the Multi-Class datasets of all sizes, the values were also constant for all WEs and equal to 0.93 for precision and 0.92 for F1-Scores and recalls. Therefore, the GBCs appeared to be insensitive to the type of WE employed for the studied datasets.

Concluding Section 3.3.1, the comparison of word embeddings across different classifiers revealed notable variations in their effectiveness. TF-IDF performed the worst for most classifiers, except for SVC. On the other hand, BERT-PCA appeared as a strong performer, especially for multiclass classification tasks. Word2Vec-PCA and FastText-PCA demonstrated best results for LR and SGD. Moreover, DTs, KNN and Gradient Boosting classifiers were mostly insensitive to the type of WE, except in the largest multiclass dataset, where BERT-PCA excelled. These findings emphasize the importance of selecting appropriate WEs based on both the classification model and dataset complexity.

3.3.2. Assessment of the PCA method for feature engineering

In this subsection, the proposed PCA-based feature engineering approaches are evaluated based on the classifiers' performance. Recall that for Word2Vec, FastText, BERT and Doc2Vec, two ways of feature engineering are applied to determine a better approach in each case. Namely, for Word2Vec and FastText, we compare taking the arithmetic mean of vectors which is the more popular practice to using PCA which attempts to maximise the amount of information extracted from the words' vectors. Moreover, for BERT, PCA is applied to reduce the dimension of pooled averages of word vectors from 384 to 50. In addition, we apply PCA to Doc2Vec document vectors, where the dimension is reduced from 300 to 100, to investigate its usefulness in comparison to taking the entire Doc2Vec document vector.

Figs. 11 and 12 compare the average F1-Score values over the seven classifiers, for each studied dataset, for Word2Vec-Average vs Word2Vec-PCA, FastText-Average vs FastText-PCA, BERT-Average vs BERT-PCA and Doc2Vec vs Doc2Vec-PCA, respectively. As shown in Fig. 11(a), for Word2Vec, the mean performance of classifiers was better for PCA than for averages for all datasets except one ("Multi 10K") where the mean F1-Score was only marginally worse for PCA. Similarly, for FastText, the mean performance of classifiers was consistently better for PCA than for taking averages, for every dataset, as shown in Fig. 11(b). Here, the relative improvement in the mean F1-Score while using PCA instead of averages ranged between 3.5 % and 12 %. In most individual cases, PCA performed better than averages for Word2Vec and FastText. For example, for the Radical Binary dataset with $n = 1000$, the F1-Score for LR with Word2Vec-Average was 0.6 while with Word2Vec-PCA it was 0.93 and the improvement was even bigger for FastText where the F1-Score for taking Average was 0.42 while for PCA it was 0.9.

For BERT, when using features of a reduced dimension via PCA, the mean F1-Scores were better than or roughly equal to the F1-Scores when taking the entire 384-dimensional vectors for six out of nine datasets, as can be seen in Fig. 12(a). However, in three cases, using the non-reduced features performed better than PCA, on average. For example, for SGD with BERT-Average the F1-Score was 0.75 and with BERT-PCA it was 0.53.

Finally, the comparison in Fig. 12(b) reveals that for Doc2Vec, applying dimension reduction using PCA gave better mean F1-Scores only for three datasets and in the remaining six cases, taking the entire original Doc2Vec vector gave slightly better results, on average. This indicates

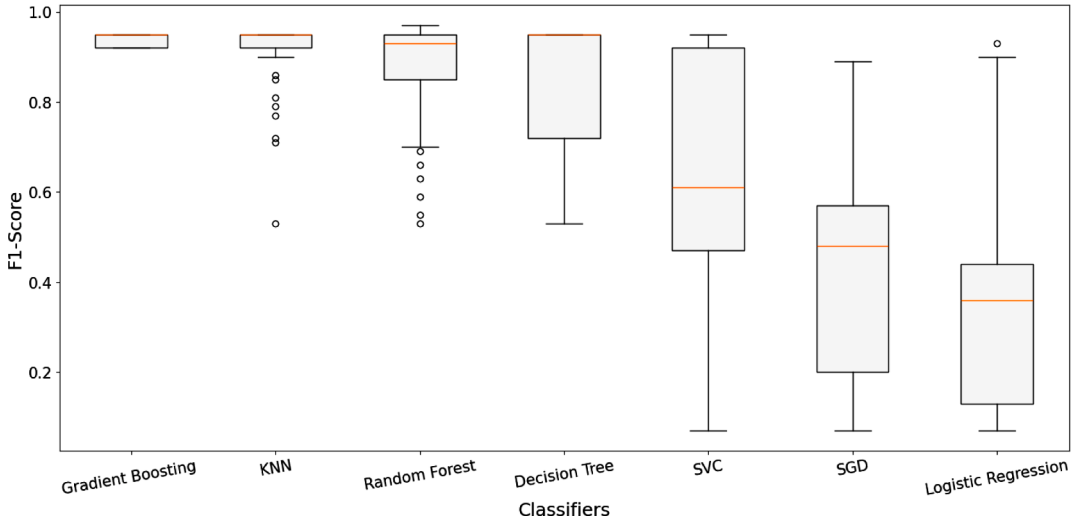


Fig. 15. Classifiers' performance comparison showing box plots of F1-Scores based on 63 values over the nine datasets and seven WEs, for each classifier, respectively.

that all the components of the Doc2Vec vectors carried useful information for the classification task.

Fig. 13 shows the violin charts and box-and-whisker plots for the overall distributions of F1-Score values over 63 cases: 9 datasets and 7 classifiers, for Word2Vec-Average vs Word2Vec-PCA, FastText-Average vs FastText-PCA, BERT-Average vs BERT-PCA and Doc2Vec vs Doc2Vec-PCA, respectively.

Based on these graphs, for Word2Vec and FastText, similar trends can be observed with the distributions of F1-Scores having longer lower tails when using averages in comparison to PCA and with considerably lower first quartiles when taking average (0.47 vs 0.66 for Word2Vec and 0.5 vs 0.66 for FastText), while the third quartiles were the same. The medians were slightly lower for PCA: 0.92 with averages vs 0.89 with PCA for Word2Vec and 0.92 with averages vs 0.91 with PCA for FastText. However, the opposite trend was observed for the mean values, which were markedly higher for PCA: 0.72 with averages vs 0.76 with PCA for Word2Vec and 0.71 with averages vs 0.77 with PCA for FastText. Generally, for Word2Vec and FastText, the overall distributions of the F1-Scores for PCA had a smaller range and were generally concentrated on higher values than for averages. For BERT and Doc2Vec, the distributions of F1-Scores for averages are very similar to the scores for PCA.

In summary, for Word2Vec and FastText, our proposed PCA approach of combining word vectors using the first principal component shows clear advantages in performance over the traditional approach of taking the average. For BERT, the advantage of lowering the dimension of averaged vectors is less marked and generally the F1-Scores are similar for both approaches. Finally, for Doc2Vec, taking the entire WE vector tends to give slightly better F1-Scores than the PCA approach. Hence, when used for dimension reduction, the number of principal components could be tuned, for example via cross-validation, which, however, adds to the computational complexity of the task.

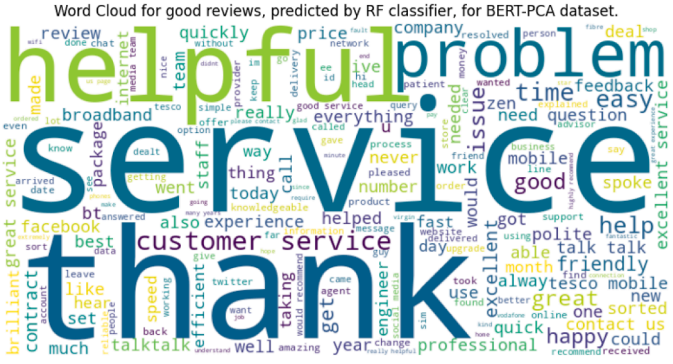
3.3.3. Comparison between classifiers

In this subsection, we comment on the overall performance of classifiers on all datasets and WEs considered. Fig. 14 visualizes the performance comparison between the seven classifiers for each dataset with bars showing the mean F1-Score over all WEs for each classifier. Moreover, Fig. 15 shows the box plots for the distributions of F1-Scores based on 63 values over the nine datasets and seven WEs, for each classifier, respectively.

The charts reveal two classifiers with a consistently high performance: GBC and KNN. GBC had the same F1-Score value of 0.95 for all



(a) Word cloud for predicted negative reviews



(b) Word cloud for predicted positive reviews

Fig. 16. Word clouds of most repeated words for negative reviews (top graph) and positive reviews (bottom graph) as predicted by the Random Forest classifier using the BERT-PCA word embedding for Radical Binary data of size 10,000.

WEs and all sample sizes for Radical Binary and Mixed Binary datasets and the value of 0.92 for the Multi-Class datasets. For Radical Binary and Mixed Binary datasets, the same pattern occurred for KNN and DT, however for the Multi-Class datasets their scores were not as high as for GBC. Therefore, the GBC classifiers proved to be the most accurate even for the more challenging data with five classes. The KNN classifier was the second best after GBC, with lower scores occurring only for the

Multi-Class dataset of size 10,000 for all WEs, ranging between 0.53 for TF-IDF and 0.9 for BERT-PCA.

The third best classifier was Random Forest (RF) with consistently high F1-Scores achieving a median of 0.93, mean of 0.87 and a narrow inter-quartile range of 0.1 indicating its stable performance over various WEs. Even though the RF was only slightly less accurate than the DT for the Radical Binary and Mixed Binary data, it outperformed the DT by a significant margin for the more challenging Multi-Class dataset achieving the mean F1-Score of 0.76 and median of 0.77 for this dataset, while the DT had the mean F1-Score of 0.71 and median of 0.68 in this case.

The remaining three classifiers: SVC, SGD and LR performed much worse than GBC, KNN, RFs and DTs, with inconsistent scores affected by the sample size and WEs.

The SVC classifier showed the highest variability and was the most sensitive to WEs among all other classifiers, with high accuracy in some cases and very low in others. Its F1-Scores ranged between 0.07 and 0.95, with an interquartile range of 0.45, a median of 0.61 and a mean of 0.63. Moreover, the SVC classifier stood out as performing relatively well with the TF-IDF text representation.

The Stochastic Gradient Descent classifier performed poorly on the datasets studied. It achieved a median F1-Score of 0.48 and a mean of 0.42, which are lower by 0.13 and 0.19, respectively, than for the SVC. Particularly poor performance was observed for the Multi-Class datasets. However, the SGD classifier had less variation in performance than the SVC, with an overall interquartile range of 0.37.

Lastly, the Logistic Regression (LR) model achieved an overall mean and median F1-Scores of 0.36, which is the lowest value among all employed classifiers. However, high scores were observed occasionally, with the maximum F1-Score of 0.93 using Word2Vec-PCA and 0.9 using FastText-PCA for Radical Binary dataset of size $n = 1000$. Like for other classifiers, the poorest performance was observed for the Multi-Class dataset. In this case, the LR model failed to produce an effective discrimination rule and had F1-Scores close to zero in many cases, with a mean F1-Score of 0.14 and a median of 0.09. Like SGD and SVC, the LR classifier also had high variation in its performance showing relatively high sensitivity to the WEs employed. Overall, LR proved to be the weakest classifier for the considered datasets.

In conclusion, the GBC and KNN classifiers demonstrated overall very high scores and stable performances and they were not particularly sensitive to WEs. RFs and DTs had slightly lower but also quite high scores and good overall performance. However, DTs performed better for simple binary datasets and were significantly less accurate for the multiclassification tasks. SVCs had poorer performance and stood out as particularly variable and sensitive to WEs. Lastly, SGD and LR performed very poorly and achieved the worst scores of all the considered classifiers, in almost all cases.

3.3.4. Illustration of a qualitative interpretation

This subsection illustrates how the results of the proposed methodology can be used in practice by telecom providers to gain valuable insights into their customers' views. A qualitative interpretation can be obtained by analyzing word clouds that visualize the most common words, with the size of each word reflecting its frequency in the texts.

As an illustration, Fig. 16 shows word clouds for the predicted negative and positive reviews, respectively based on the Random Forest classifier and BERT-PCA word embedding, for the Radical-Binary dataset of size $10,000$. The word cloud for negative reviews reveals a strong presence of terms such as "call", "time", "customer service" "contract," "told", "said", "issue" and "problem". These words indicate dissatisfaction related to customer support, communication delays, and contractual disputes. Repeated use of temporal terms like "month," "day," "hour," and "time" reflect frustrations about delays and waiting periods, reinforcing themes of inefficiency and unresponsiveness. In contrast, the word cloud for positive reviews is strongly dominated by the words "thank", "helpful", "service", "problem" and with presence of words such as "quickly", "easy", and "friendly" and "customer service". This suggests that positive reviews often praise the quality of assistance, speed of resolution, and attitude of the staff. Positive temporal phrases such as "quickly" and "efficient" also appear frequently, indicating satisfaction with prompt support. Interestingly, the word "problem" also appears in positive reviews, but in a different context-likely referring to problems that were successfully resolved.

Overall, the word clouds for subgroups of customers' comments, categorized by the predicted star ratings, offer a visual validation of the classifier's ability to distinguish between sentiment polarities. They also provide insight into the most frequent themes customers associate with satisfaction and dissatisfaction, which can inform both business intelligence and model interpretability.

3.4. Time - classifiers' performance analysis

In this subsection, we explore the relationship between time and F1-Score for the considered models and scenarios with the aim of assessing their efficiency and identifying the most and the least efficient approaches. Fig. 17 shows a scatter plot of classifiers' fitting times versus

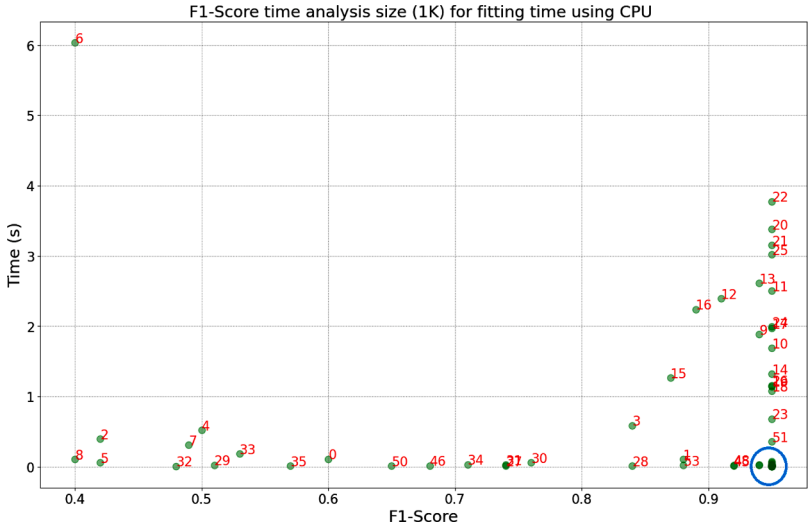


Fig. 17. Training times against F1-Scores for classifiers trained using CPU, for the Mixed-Binary dataset of size 1,000, where the blue circle refers to overlapping points: 36–44, 47, 49, 52, 54–62, as listed in Table 1.

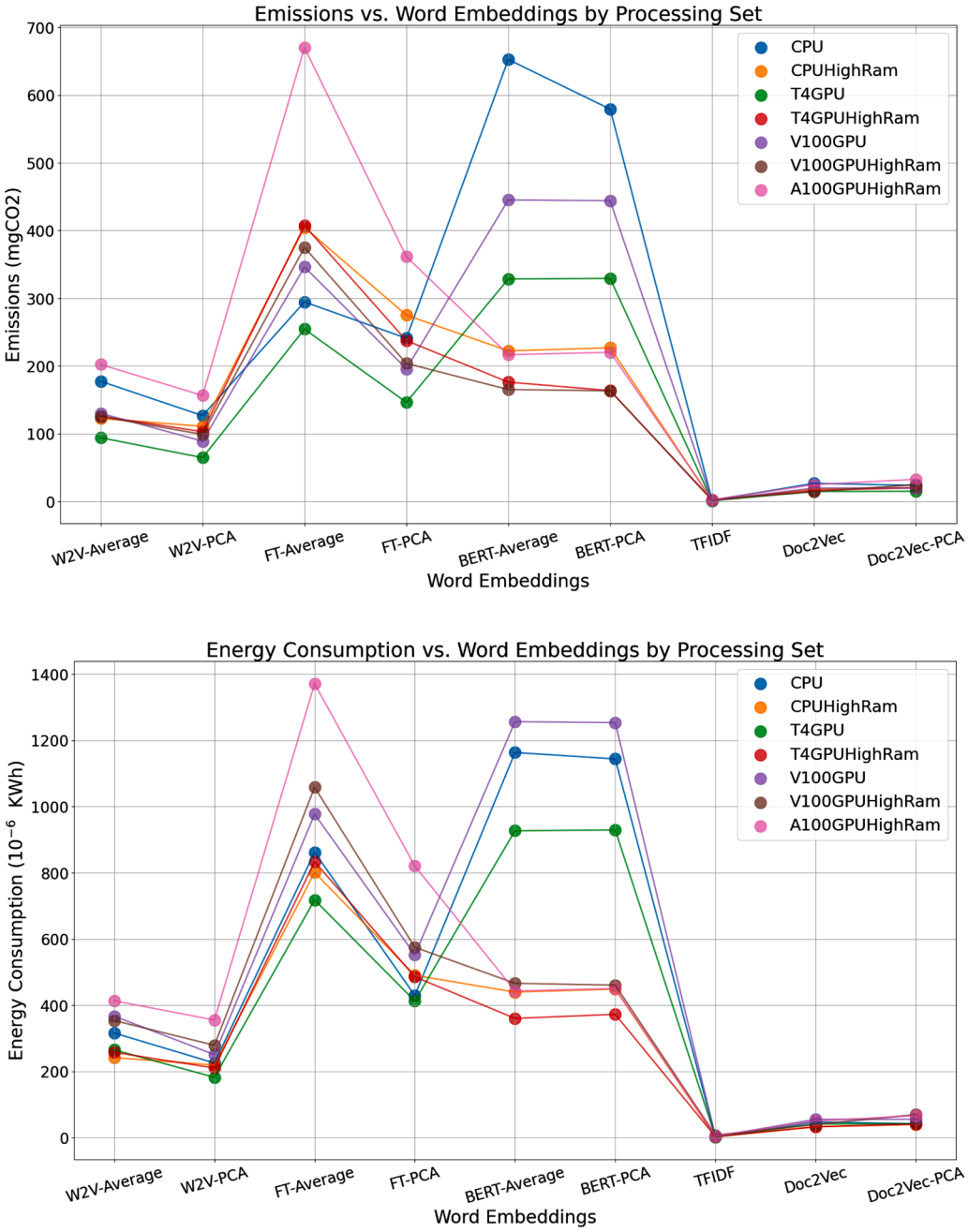


Fig. 18. Emissions and Energy Consumed vs Word Embedding graphs for Feature Extraction using Mixed-Binary dataset of size 1,000, for several computing sets.

F1-Scores for 63 cases, using the nine WEs and seven classifiers, for the Mixed Binary data of size 1,000, as an example, as very similar patterns occurred for all the other datasets. Each point is annotated by its index which can be interpreted using Table 1, where the name of the WE and the classifier, as well as the fitting time, precision, recall and F1-Score values are listed.

The scatter plot reveals that many points have a very high F1-Score values close to 1 for a wide range of fitting times. On the other hand, many points have a very low fitting time for a wide range of F1-Scores. For example, for the Mixed Binary dataset, classifiers with fitting times below 0.11s have F1-Scores varying between 0.4 and 0.95. These patterns indicate that, overall, there is no straightforward or linear relationship between the fitting times and classifiers' performance.

However, some positive correlation can be observed. Gradient Boosting classifiers (points marked with numbers 20, 21, 22, 25), as well as Random Forests (points marked with numbers 11, 13) had high performance coupled with relatively long fitting times and Logistic Regression (points 2, 5, 8) had low fitting times and low performance.

We note an outlying point (number 6) associated with very low performance metrics and the longest fitting time that can be observed in Fig. 17. This case refers to using TF-IDF with a LR classifier. This specific pair of TF-IDF coupled with LR had the same pattern of relatively high time consumption with low accuracy value also for other computing powers and data sizes indicating the worst overall efficiency.

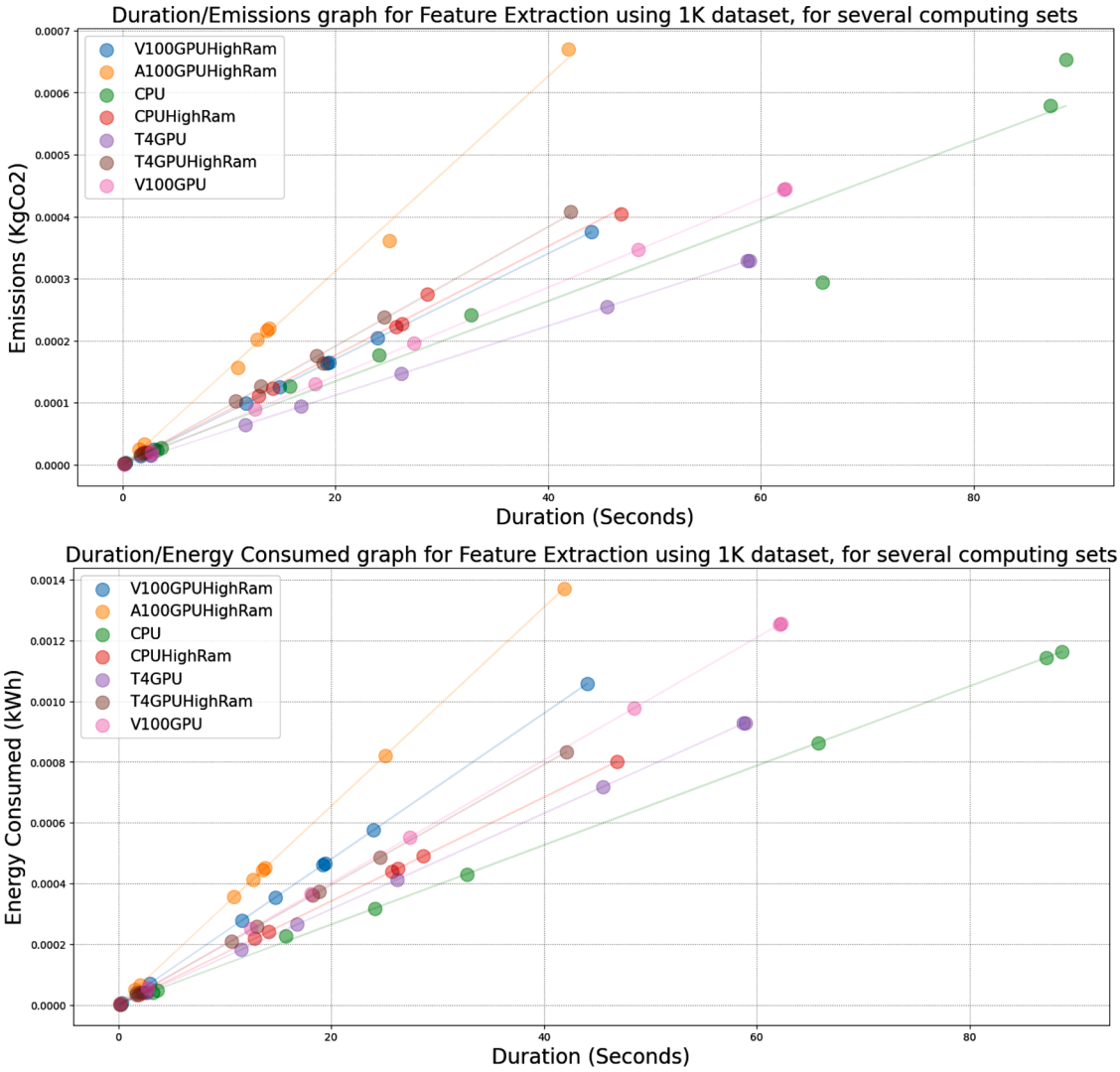


Fig. 19. Duration/Emissions and Energy Consumed graph for Feature Extraction using Mixed-Binary dataset of size 1,000, for several computing sets, with added least squares lines.

The best efficiency scenario is characterized by a short fitting time and high F1-Score. For the Mixed-Binary data, this can be observed for the KNN and DTs classifiers (points clustered in the lower right corner of the graph in Fig. 17 marked with numbers 36 - 44, 47, 49, 52, 54 - 62), while for the Multi-Class dataset KNN is the most efficient, closely followed by most SVC classifiers.

In conclusion, we find that the least efficient classifier was Logistic Regression applied to a TF-IDF text representation which had relatively long fitting times and poor performance, while the most efficient one was KNN with the fastest training times coupled with the best performance metrics irrespective of the WE used. Moreover, the relationship between the consumed time and performance measures can be described by a low to moderate positive correlation, indicating a lack of a simple or strong pattern for the relationship between the computational intensity and the quality of the resulting classification rule.

3.5. Estimation of energy consumption for word embeddings

In this subsection, the environmental impact of the considered text representation methods is explored by estimating energy consumed and CO2 emissions for various processing setups. The use of machine learning models that run on modern computing systems such as cloud computing and parallel processing units including the use of GPUs, CPUs and TPUs, requires significant amounts of energy. Some estimates of the electricity consumption of data centers are of the order 2 % of the total energy generated in the world (Freitag et al., 2021; Terenius et al., 2023). The large use of energy required in machine learning and AI may have adverse environmental effects. Some countries have introduced regulations, because of the concern about the substantial use of electricity by data centers, and approval for building new data centers has sometimes been refused, because of concerns about the resources used. For example, the European Commission requires Data Centers to report on sustainability targets, including electricity use.

This motivates studying the energy used in machine learning. The aim of this subsection is to illustrate energy consumption and CO2 emissions for the WEs employed in this study, for various computing settings and identify the most and the least environment friendly cases.

There have been some discussions in the scientific community on ways to study machine learning algorithms that also consider energy consumption (Lottick et al., 2019; Mehlin et al., 2023). The paper by Bannour et al. (2021) reviews energy consumption of NLP methods. Understanding their respective contributions to emissions is important in order to highlight the cases of unnecessary data storing and processing and their deep effects on the environment, in

Table 1
Training times, precisions, recalls and F1-Scores for classifiers trained using CPU for Mixed Binary data of size 1,000, as plotted in Fig. 17.

Index	Classifier	Word embedding	Time (sec)	Precision	Recall	F1-Score
0		W2V-Average	0.11	0.6	0.7	0.6
1		W2V-PCA	0.11	0.91	0.88	0.88
2		FT-Average	0.39	0.38	0.56	0.42
3	Logistic	FT-PCA	0.59	0.89	0.85	0.84
4	Regression	BERT-Average	0.52	0.55	0.62	0.5
5		BERT-PCA	0.06	0.38	0.56	0.42
6		TF-IDF	6.03	0.32	0.55	0.4
7		Doc2Vec	0.31	0.5	0.61	0.49
8		Doc2Vec-PCA	0.11	0.37	0.55	0.4
9		W2V-Average	1.88	0.96	0.95	0.94
10		W2V-PCA	1.69	0.96	0.95	0.95
11		FT-Average	2.5	0.96	0.96	0.95
12	Random	FT-PCA	2.39	0.92	0.91	0.91
13	Forest	BERT-Average	2.61	0.95	0.94	0.94
14		BERT-PCA	1.32	0.97	0.95	0.95
15		TF-IDF	1.26	0.88	0.87	0.87
16		Doc2Vec	2.24	0.9	0.89	0.89
17		Doc2Vec-PCA	1.97	0.97	0.95	0.95
18		W2V-Average	1.08	0.97	0.95	0.95
19		W2V-PCA	1.15	0.97	0.95	0.95
20		FT-Average	3.38	0.97	0.95	0.95
21	Gradient	FT-PCA	3.15	0.97	0.95	0.95
22	Boosting	BERT-Average	3.77	0.97	0.95	0.95
23		BERT-PCA	0.68	0.97	0.95	0.95
24		TF-IDF	1.99	0.97	0.95	0.95
25		Doc2Vec	3.02	0.97	0.95	0.95
26		Doc2Vec-PCA	1.16	0.97	0.95	0.95
27		W2V-Average	0.01	0.74	0.8	0.74
28		W2V-PCA	0.01	0.9	0.85	0.84
29		FT-Average	0.02	0.51	0.62	0.51
30	Stochastic	FT-PCA	0.06	0.86	0.79	0.76
31	Gradient	BERT-Average	0.03	0.74	0.8	0.74
32	Descent	BERT-PCA	0.01	0.44	0.6	0.48
33		TF-IDF	0.18	0.48	0.65	0.53
34		Doc2Vec	0.03	0.68	0.78	0.71
35		Doc2Vec-PCA	0.01	0.53	0.67	0.57
36		W2V-Average	0.01	0.97	0.95	0.95
37		W2V-PCA	0.01	0.97	0.95	0.95
38		FT-Average	0.03	0.97	0.95	0.95
39	Decision	FT-PCA	0.03	0.97	0.95	0.95
40	Tree	BERT-Average	0.04	0.97	0.95	0.95
41		BERT-PCA	0.01	0.97	0.95	0.95
42		TF-IDF	0.06	0.97	0.95	0.95
43		Doc2Vec	0.03	0.97	0.95	0.95
44		Doc2Vec-PCA	0.01	0.97	0.95	0.95
45		W2V-Average	0.01	0.93	0.92	0.92
46		W2V-PCA	0.02	0.79	0.7	0.68
47		FT-Average	0.02	0.97	0.95	0.95
48	Support	FT-PCA	0.02	0.93	0.92	0.92
49	Vector	BERT-Average	0.02	0.95	0.94	0.94
50	Classifier	BERT-PCA	0.01	0.78	0.68	0.65
51		TF-IDF	0.36	0.97	0.95	0.95
52		Doc2Vec	0.03	0.96	0.94	0.94
53		Doc2Vec-PCA	0.02	0.9	0.88	0.88
54		W2V-Average	0.01	0.97	0.95	0.95
55		W2V-PCA	0.01	0.97	0.95	0.95
56		FT-Average	0.01	0.97	0.95	0.95
57	K-Nearest	FT-PCA	0.01	0.97	0.95	0.95
58	Neighbours	BERT-Average	0.01	0.97	0.95	0.95
59		BERT-PCA	0.0	0.97	0.95	0.95
60		TF-IDF	0.08	0.97	0.95	0.95
61		Doc2Vec	0.01	0.97	0.95	0.95
62		Doc2Vec-PCA	0.0	0.97	0.95	0.95

accordance with the recommendation of García-Martín et al. (2019), as more GPU usage is needed globally due to expanding applications of machine learning methods. Lee and Brooks (2006) presented a model to calculate the power consumed using regression analysis applied to a micro-architectural dataset. Laurenzano et al. (2014) built an application for HPC benchmarks to calculate performance and energy by measuring wall power. Mazouz et al. (2017) presented a methodology to calculate the consumed energy of hardware processors, while García-Martín et al. (2019) monitored many experiments to measure the estimated energy consumption by machine learning kernels.

In this short study, we use the Python library CodeCarbon (Courty et al., 2024) to measure the amount of energy and CO2 emissions, for various processing sets, including: T4-GPU-HighRAM, CPU-HighRAM, CPU, T4-GPU, A100-GPU-HighRAM, V100-GPU, V100-GPU-HighRAM.

The CO2 emissions obtained from the CodeCarbon library[3] are computed from the product of the energy consumed and a constant factor. The constant factor depends on the mixture of energy generation methods, such as coal, natural gas, and renewable sources. The constant factor to compute the CO2 emissions depends on the country, because, for example, some countries use more coal to generate electricity than others, and it is encoded in a lookup table.

Fig. 18 shows produced CO2 emission and consumed energy vs Word Embedding, where each line reflects a different processing set for the Mixed-Binary dataset of size 1,000, as an example. These values are the average of ten trials for each of the seven processing sets. Moreover, Fig. 19 shows scatter plots of duration versus CO2 emission and duration versus energy consumed, respectively.

3.5.1. *Comparison of word embeddings for a fixed processing set*

Fig. 18 reveals that for the four computing settings that use HighRAM (T4-GPU-HighRAM, CPU-HighRAM, A100-GPU-HighRAM, V100-GPU-HighRAM), FastText-Average WEs consumed the most energy and had the highest CO2 emissions. For example, for the T4-GPU-HighRAM processing set, FastText-Average had approximately 4 times more emissions and energy consumption than W2V-PCA and around 25 times more than Doc2Vec. FastText-PCA was the second most energy-consuming and CO2-emitting WE for HighRAM computing settings, with an energy consumption around 60 % of that needed by FastText-Average. On the other hand, for the three processing settings that do not use HighRAM (CPU, T4-GPU and V100-GPU), BERT-Average and BERT-PCA were the two most energy-consuming and CO2-emitting WEs, needing around 1.3 times as much energy as FastText-Average, which had the second highest energy consumption and CO2 emissions, around 3.7 times as much as W2V-Average and 25 times as much as Doc2Vec.

On the other side of the spectrum, TF-IDF used the least energy and had the lowest CO2 emissions for all processing settings, by two orders of magnitude in most cases. For example, for T4-GPU-HighRAM, TF-IDF had around 260 times less emissions and energy consumption than FastText-Average. This can be explained by the nature of TF-IDF, as even though it contains a large number of columns, the majority of them are null values and hence it is stored in the memory as sparse matrices and given that we used short texts in this study the TF-IDF process is quite energy efficient. The second most environmentally friendly WE was Doc2Vec and it was closely followed by Doc2Vec-PCA. These two WEs stood out from others by a significant margin, having at least an order of magnitude less usage of energy and CO2 emission. Lastly, the two Word2Vec WEs can be described as medium in their energy consumption and CO2 emissions compared to the others.

For a fixed processing set, the time consumption was strongly positively correlated with energy and emissions, respectively, as illustrated in Fig. 19. Hence, the WE with the shortest time always had the lowest energy consumption and CO2 emissions and vice versa. However, the

[3] https://mlco2.github.io/codecarbon/methodology.html

magnitude of differences between WEs varied according to the slope coefficient, which was different for each processing set, so that, for example, having a double time would not necessarily be related with a double energy consumption or emissions.

3.5.2. Comparison of processing sets

Changing the processing set also significantly affected CO2 emissions and energy consumed.

For Word2Vec and FastText (both, Average and PCA approaches), T4-GPU led to the lowest CO2 emissions and energy consumption among all other processing sets, while A100-GPU-HighRAM - the highest. A similar pattern occurred for TF-IDF, as T4-GPU was still the most environmentally friendly. However, in this case V100-GPU-HighRAM led to the highest CO2 emissions and energy consumption, with A100-GPU-HighRAM being the second highest. Also, for Doc2Vec and Doc2Vec-PCA, T4-GPU was related to the lowest CO2 emissions and CPU-HighRAM - the lowest energy consumption.

For BERT, the highest CO2 emissions and the most energy consumption were observed for CPU and V100-GPU. On the other hand, the lowest CO2 emissions and the least energy consumption occurred for V100-GPU-HighRAM and T4-GPU-HighRAM. Generally, BERT-based WEs showed a different pattern from the other WEs with respect to the processing sets' effects, in that HighRAM settings lead to a more environmentally friendly performance than non-highRAM settings. By comparing two related sets: CPU and CPU-HighRAM, we notice that the CPU-HighRAM values for Duration, Emission and Energy are always less than the corresponding CPU values and the same pattern holds for T4-GPU against T4-GPU-HighRAM and V100-GPU against V100-GPU-HighRAM. For example, for BERT-Average, using the CPU processing set needed three times the amount of energy that is needed when using CPU-HighRAM and twice the amount of emission.

However, for other WEs this pattern does not always hold. In particular, for FastText-Average, FastText-PCA and TF-IDF, HighRAM settings usually have higher CO2 emissions and energy consumption. It is also important to note that, in some cases, shorter time does not lead to smaller energy and CO2 emissions. For FastText-PCA and TF-IDF, although the task was completed for CPU-HighRAM in a shorter time than for CPU, the CO2 emissions and energy consumption were higher for CPU-HighRAM.

The scatter plots in Fig. 19 show that changing the processing set leads to a change in the slope of the best-fit line, which reflects the rate of change of emission or energy, with time. Processing sets that usually need more power like GPU and HighRAM, have a higher slope than standard processing sets like CPU.

For a fixed amount of time consumption, A100-GPU-HighRAM lead to the most energy consumed and the highest CO2 emissions, which is demonstrated in Fig. 19 by the yellow line of best fit being above all other lines in both scatter plots. T4-GPU-HighRAM had the second highest emissions (brown line in the top scatter plot), while V100-GPU-HighRAM had the second highest energy consumption (blue line in the bottom scatter plot). On the other hand, T4-GPU had the lowest CO2 emissions (purple line in the top plot) and CPU had the lowest energy consumption (green line in the bottom plot) for a fixed duration.

Concluding Section 3.5, the presented results emphasize the environmental impact of computations and the need for more energy-saving models, as different processing sets and WEs lead to different values of produced emissions and consumed energy, which has a potential impact on the Earth's future. We found that TF-IDF and Doc2Vec are the most energy efficient embeddings, while FastText-Average and BERT-Average and PCA models are the most resource-consuming. The results and findings offer a guidance for researchers in telecom and similar fields who seek to optimize classification accuracy with model efficiency. Our results inform model selection under resource constraints, which is important in edge computing or real world systems. However, WE models require a trade-off between computational efficiency and classification accuracy, as faster models, such as TF-IDF, often lack the depth of contextual understanding compared to more sophisticated models that need more time to be trained, as seen in Section 3.4. Thus, when selecting an embedding approach, one must carefully balance speed, energy consumption and accuracy, depending on the specific requirements of the application.

4. Conclusion

This paper investigates the effectiveness of several word embedding techniques in classifying telecom customer reviews by star ratings. The results show that BERT combined with PCA-based dimensionality reduction generally outperforms other considered word embeddings, especially for the more challenging multiclass tasks. Moreover, PCA-based feature engineering proved to be superior to traditional averaging methods for Word2Vec and FastText. Among classifiers, Gradient Boosting and KNN demonstrated the most accurate predictions and stable performance. Additionally, the study highlights trade-offs between accuracy, training time, and energy consumption across word embedding and classifier choices.

Energy consumption analysis revealed that computational efficiency varies significantly among embedding techniques, with TF-IDF being the most resource-efficient and FastText the most computationally demanding. The findings emphasize the importance of selecting the appropriate WE based on classification complexity, model compatibility and computational constraints. A general overview of the methods employed in this study is presented in Appendix C.

As with many data science projects, this study was performed using a Jupyter notebook and input from csv files. Now that a successful proof of concept has been achieved and the best NLP methods are found, the project could be moved to the deployment stage. The Python in the notebook could be refactored into Python scripts run inside containers and called via web services such as REST APIs. Data could be pulled in from the TrustPilot API, and other sources of free text from customers that a company may have. The system could then provide reports or dashboards that would provide real time monitoring of the star classification of the free text written by telecom customers. The final system would give real-time feedback to the company on its customers and thus potentially help reduce the number of customers not renewing their contracts.

While this study is primarily empirical, it contributes a thorough comparative understanding of embedding-classifier configurations, which is necessary groundwork for practical model deployment in telecom NLP. Compared to existing studies that either focus on very few word embeddings or limit the analysis to accuracy metrics, our approach is broader in scope and provides detailed comparison across several classifiers and evaluation aspects including time- and energy consumption. Examples of recent benchmarks obtained in similar studies are shown in Appendix B.

However, this study is limited by relying on popular word embedding models without fine-tuning, and it does not provide new architectures. Also, it is important to note that all the results and conclusions are limited to the considered datasets, and may differ for other data sources or applications. Moreover, only reviews written in English were considered, and the models may reveal different performance patterns with other languages. Finally, we measured energy use for a limited scope of sample sizes and the conclusions may not apply when using bigger systems.

In the future, it is important to perform similar comprehensive tests in other areas such as healthcare or finance, and also include texts written in different languages. More advanced word embeddings and language models could also be considered to better handle complex language patterns and provide richer contextual understanding for classification tasks. Moreover, other ways to reduce data dimensionality such as UMAP or autoencoders, which may yield better results than PCA, should be examined. Future studies should also look at how model size, training time, and energy use are connected, to make machine learning and AI more environmentally friendly.

CRediT authorship contribution statement

Hesham Abdelmotaleb: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data Curation, Writing – original draft, Visualization, Project administration.; **Craig Mcneile:** Supervision, Validation, Writing – review & editing, Methodology, Resources.; **Małgorzata Wojtyś:** Supervision, Writing – review & editing, Methodology, Resources.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The Authors sincerely thank the Associate Editor and Reviewers whose valuable feedback has substantially contributed to improving the manuscript.

Appendix A. Classifiers' settings for Section 2.3.3

The classifiers were imported in Python language from the Sklearn library, using the following parameters, respectively:

- Logistic Regression: *penalty = l2, solver = 'sag', C = 1.0, fit_intercept = True, solver = 'lbfgs', max_iter = 100*
- Decision Tree: *criterion = 'gini', max_depth = 3, splitter = 'best', min_samples_split = 2, min_samples_leaf = 1*
- Random Forest: *criterion = 'gini', n_estimators = 300, max_depth = 7, min_samples_split = 2, min_samples_leaf = 1, min_weight_fraction_leaf = 0.0*
- Gradient Boosting : *n_estimators = 100, max_depth = 3, loss = 'log_loss', learning_rate = 0.1, subsample = 1.0, criterion = 'friedman_mse'*
- Stochastic Gradient Descent: *penalty = 'l2', loss = 'log', learning_rate = 'optimal', alpha = 0.0001, l1_ratio = 0.15, fit_intercept = True*
- Support Vector Machine: *kernel = 'rbf', max_iter = 100, C = 1.0, gamma = 'auto', degree = 3, coef0 = 0.0*
- K-Nearest Neighbour: *n_neighbors = 5, weights = 'uniform', algorithm = 'auto', leaf_size = 30, p = 2, metric = 'minkowski'*

Appendix B. Recent baselines studies

To position the paper within the context of existing academic literature, Table B.2 provides an overview of several recent studies that applied various word embedding techniques for text classification tasks. These studies used a variety of datasets, including Arabic sentiment datasets, English sentiment and news datasets, as well as specialized domains like Sinhala news or spam detection. The table compares the word embedding methods, classifiers used, and their reported performance.

Based on the table, it is clear that combining modern word embedding models such as BERT or FastText with deep learning classifiers like BiLSTM or CNN often leads to strong performance. For example, Sabbeh and Fasihuddin (2023) achieved an F1-score of 94.5 % using AraBERT with BiLSTM on Arabic sentiment datasets. Similarly, Wang et al. (2020) reported high F1-scores on SST2 with both BiLSTM and CNN using GloVe and BERT embeddings. These results offer useful baselines for evaluating new models or embedding strategies in future work.

Appendix C. Summaries of the employed methods

In this subsection, a general overview of the methods employed in this study is presented.

Table C.3 evaluates the word embedding techniques considered in this research, including the PCA-based feature engineering techniques. BERT-PCA emerged as the best performing technique, especially for multi-class classification tasks, offering a balance between dimensionality, speed, and accuracy. Word2Vec and FastText with PCA also performed well, particularly with simpler classifiers like LR and SGD. Doc2Vec and its PCA variant were fast and efficient but had lower accuracy in more complex settings.

Table B.2
Selected studies on text classification using word embeddings. These serve as baselines in terms of model configuration and datasets used.

Study	Dataset	Word Embedding	Classifiers	Key results
(Sabbeh & Fasihuddin, 2023)	Arabic sentiment HARD, Khooli ArSASm ASTD	AraBERT, Word2Vec, FastText	CNN, BiLSTM	BERT + BiLSTM F1 : 94.50 %
(Wang et al., 2020)	English sentiment SST2, AAPD 20News, Reuters	GloVe, FastText, BERT, ELMo	CNN, BiLSTM	Macro F1: SST2 + BiLSTM: 90 % SST2 + CNN: 90 %
(Lakmal et al., 2020)	Sinhala news	FastText, GloVe, W2V CBOW, W2V skip-gram	RNN-LSTM	F1 : 87.76 % Sentiment Analysis & FastText
(Occhipinti et al., 2022)	Enron Spam SMS, Email	TF-IDF, GloVe	SVM, KNN LR, RF MPNN	F1: 94 % Random Forest

Table C.3
Overall comparison of Word Embedding techniques used in the study.

Embedding	Dim.	Time cost	Advantages & limitations
Word2Vec-Average	100	Low	Good semantic capture; fast training; poor on small datasets and ignores word order.
Word2Vec-PCA	100	Low	Improved performance over average; PCA weights informative terms better; slight increase in complexity.
FastText-Average	300	High	Captures sub-word info; handles OOV words; slow and sometimes inconsistent performance.
FastText-PCA	300	High	Slightly better than average; time-consuming; less effective than BERT or Doc2Vec in complex tasks.
BERT-Average	384	Medium	Contextual; bidirectional; accurate but slow and memory-intensive; weak on small datasets.
BERT-PCA	50	Medium	Reduced size, high accuracy, especially for multi-class tasks; fastest BERT variant; best trade-off.
TF-IDF	-	Very Low	Simple and fast; does not capture semantics; performs poorly with most classifiers but surprisingly well with SVC.
Doc2Vec	300	Low	Captures document-level semantics; fast and simple; lower performance in multiclass.
Doc2Vec-PCA	100	Low	Slight improvement over full vector; limited boost; may lose useful features.

Table C.4
Overall comparison of classifiers used in the study.

Classifier	Time cost	Sensitivity	Summary of performance and notes
Logistic Regression (LR)	Medium	High	Weakest overall; only strong with W2V-PCA and FT-PCA on small binary sets.
Random Forest (RF)	Medium	Medium	High performance especially with BERT-PCA and Doc2Vec-PCA; reliable and interpretable.
Gradient Boosting (GBC)	High	Low	Best accuracy overall, consistent across datasets; good on all embeddings; slow to train.
Stochastic Gradient Descent (SGD)	Very Low	High	Strong with W2V-PCA and FT-PCA; poor elsewhere; unstable in multiclass tasks.
Decision Tree (DT)	Very Low	Very Low	Identical performance across WEs in binary data; weak in multiclass; prone to overfitting.
Support Vector Classifier (SVC)	Medium	High	Performs best with TF-IDF; inconsistent with others; sensitive to embedding and overfits.
K-Nearest Neighbors (KNN)	Low	Low	Fastest training; high accuracy; insensitive to embedding; performs well in binary and multiclass.

Table C.4 compares classifiers used in this study, showing the Gradient Boosting and KNN to be the preferred algorithms, combining accuracy and robustness. Random Forest also showed stable performance, particularly when used with BERT-PCA and Doc2Vec-PCA. Logistic Regression and SGD showed high sensitivity to the embedding method and generally underperformed. Interestingly, SVC performed relatively well with TF-IDF, highlighting the importance of choosing the right classifier-embedding combination.

References

Abdelmotaleb, H., McNeile, C., & Wojtyś, M. (2023). A comparison of a novel optimized GSDMM model with K-Means clustering for topic modelling of free text. *Journal of Machine Intelligence and Data Science*, *4*, 52–62.

Anderson, T. W. (2003). An introduction to multivariate statistical analysis. (3rd ed.). Wiley.

Bannour, N., Ghannay, S., Névéol, A., & Ligozat, A. L. (2021). Evaluating the carbon footprint of nlp methods: A survey and analysis of existing tools. In *Proceedings of the second workshop on simple and efficient natural language processing* (pp. 11–21).

Binder, M., Heinrich, B., Klier, M., Obermeier, A., & Schiller, A. P. R. (2019). Explaining the stars: Aspect-based sentiment analysis of online customer reviews. *Twenty-Seventh European Conference on Information Systems (ECIS2019), Stockholm-Uppsala, Sweden*.

Carneiro, T., Da Nóbrega, R. V. M., Nepomuceno, T., Bian, G. B., De Albuquerque, V. H. C., & Reboucas Filho, P. P. (2018). Performance analysis of google colaboratory as a tool for accelerating deep learning applications. *IEEE Access*, *6*, 61677–61685.

Chang, M. W., Toutanova, K., Lee, K., & Devlin, J. (2019). Language model pre-training for hierarchical document representations. arXiv preprint: arXiv.1901.09128

Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). What does BERT look at? An analysis of BERT's attention. In T. Linzen, G. Chrupała, Y. Belinkov, & D. Hupkes (Eds.), *Proceedings of the 2019 ACL workshop blackboxNLP: Analyzing and interpreting neural networks for NLP* (pp. 276–286). Florence, Italy: Association for Computational Linguistics.

Courty, B., Schmidt, V. et al., (2024). Lucas-Otavio, Jpw, MinervaBooks mlco2/codecarbon: v2.4.1. https://doi.org/10.5281/zenodo.11171501.

Eisa, D. A., Taloba, A. I., & Ismail, S. S. (2018). A comparative study on using principle component analysis with different text classifiers. *International Journal of Computer Applications*, *180*, 1–6. https://doi.org/10.5120/ijca2018916800.

Freitag, C., Berners-Lee, M., Widdicks, K., Knowles, B., Blair, G. S., & Friday, A. (2021). The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations. *Patterns*, *2*.

García-Martín, E., Rodrigues, C. F., Riley, G., & Grahn, H. (2019). Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing*, *134*, 75–88. https://doi.org/10.1016/j.jpdc.2019.07.007.

Gupta, V., Giesselbach, S., Rüping, S., & Bauckhage, C. (2019). Improving word embeddings using kernel PCA. In *Proceedings of the 4th workshop on representation learning for NLP (RepL4NLP-2019)* (pp. 200–208).

Jolliffe, I. T. (2002). Principal component analysis. (2nd ed.). Springer-Verlag.

Kottur, S., Vedantam, R., Moura, J. M. F., & Parikh, D. (2016). Visual Word2Vec (vis-w2v): Learning visually grounded word embeddings using abstract scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*.

Lakmal, D., Ranathunga, S., Peramuna, S., & Herath, I. (2020). Word embedding evaluation for Sinhala. In *Proceedings of the twelfth language resources and evaluation conference* (pp. 1874–1881). Marseille, France: European Language Resources Association. https://aclanthology.org/2020.lrec-1.231.

Laurenzano, M. A., Tiwari, A., Jundt, A., Peraza, J., Ward, W. A., Campbell, R., & Carrington, L. (2014). Characterizing the performance-energy tradeoff of small arm cores in HPC computation. In F. Silva, I. Dutra, & V. Santos Costa (Eds.), *Euro-par 2014 parallel processing* (pp. 124–137). Cham: Springer International Publishing.

Lebret, R., & Collobert, R. (2014). Word embeddings through hellinger PCA. In S. Wintner, S. Goldwater, & S. Riezler (Eds.), *Proceedings of the 14th conference of the European chapter of the association for computational linguistics* (pp. 482–490). Gothenburg, Sweden: Association for Computational Linguistics. https://doi.org/10.3115/v1/E14-1051.

Lee, B. C., & Brooks, D. M. (2006). Accurate and efficient regression modeling for microarchitectural performance and power prediction. In *Proceedings of the 12th international conference on architectural support for programming languages and operating systems* (pp. 185 – 194). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/1168857.1168881.

Lottick, K., Susai, S., Friedler, S., & Wilson, J. (2019). Energy usage reports: Environmental awareness as part of algorithmic accountability. In *NeurIPS 2019 workshop on tackling climate change with machine learning*. https://www.climatechange.ai/papers/neurips2019/8.

Mazouz, A., Wong, D. C., Kuck, D., & Jalby, W. (2017). An incremental methodology for energy measurement and modeling. In *Proceedings of the 8th ACM/SPEC on international conference on performance engineering* (pp. 15–26). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3030207.3030224.

Mehlin, V., Schacht, S., & Lanquillon, C. (2023). Towards energy-efficient deep learning: An overview of energy-efficient approaches along the deep learning lifecycle. arXiv preprint arXiv:2303.01980

Mikolov, T., Corrado, G., Chen, K., & Dean, J. (2013). Efficient estimation of word representations in vector space. 1–12.

Monett, D., & Stolte, H. (2016). Predicting star ratings based on annotated reviews of mobile apps. IEEE. *2016 Federated conference on computer science and information systems (FedCSIS)*, pp. 421–428.

Occhipinti, A., Rogers, L., & Angione, C. (2022). A pipeline and comparative study of 12 machine learning models for text classification. *Expert Systems with Applications*, *201*, 117193. https://doi.org/10.1016/j.eswa.2022.117193.

Okkalioglu, M. (2024). A novel redistribution-based feature selection for text classification. *Expert Systems with Applications*, *246*, 123119. https://doi.org/10.1016/j.eswa.2023.123119.

Oscar Deho, B., William Agangiba, A., Felix Aryeh, L., & Jeffery Ansah, A. (2018). Sentiment analysis with word embedding. In *2018 IEEE 7th international conference on adaptive science & technology (ICAST)* (pp. 1–4). https://doi.org/10.1109/ICASTECH.2018.8506717.

Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532–1543).

Powers, D. M. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, (pp. 37–63).

Sabbeh, S. F., & Fasihuddin, H. A. (2023). A comparative analysis of word embedding and deep learning for arabic sentiment classification. *Electronics*, *12(6)*, 1425.

Sammut, C., & Webb, G. I. (Eds.) (2010). TF–IDF. Boston, MA: Springer US. 986–987, https://doi.org/10.1007/978-0-387-30164-8_832.

Saumya, S., Singh, J. P., Baabdullah, A. M., Rana, N. P., & Dwivedi, Y. K. (2018). Ranking online consumer reviews. *Electronic Commerce Research and Applications*, *29*, 78–89.

Shao, Y., Taylor, S., Marshall, N., Morioka, C., & Zeng-Treitler, Q. (2018). Clinical text classification with word embedding features vs. bag-of-words features. In *2018 IEEE international conference on big data (big data)* (pp. 2874–2878). https://doi.org/10.1109/BigData.2018.8622345.

Terenius, P., Garraghan, P., & Harper, R. (2023). A material social view on data center waste heat: Novel uses and metrics. *Frontiers in Sustainability*, *3*, 1008583.

Upadhyay, D., Manero, J., Zaman, M., & Sampalli, S. (2020). Gradient boosting feature selection with machine learning classifiers for intrusion detection on power grids. *IEEE Transactions on Network and Service Management*, *18*, 1104–1116.

Wang, C., Nulty, P., & Lillis, D. (2020). A comparative study on word embeddings in deep learning for text classification. In *Proceedings of the 4th international conference on natural language processing and information retrieval* (pp. 37–46).

Wang, Q., Liu, P., Zhu, Z., Yin, H., Zhang, Q., & Zhang, L. (2019). A text abstraction summary model based on bert word embedding and reinforcement learning. *Applied Sciences*, *9*. https://doi.org/10.3390/app9214701.

Wang, Y., Liu, S., Afzal, N., Rastegar-Mojarad, M., Wang, L., Shen, F., Kingsbury, P., & Liu, H. (2018). A comparison of word embeddings for the biomedical natural language processing. *Journal of Biomedical Informatics*, *87*, 12–20.

Yao, T., Zhai, Z., & Gao, B. (2020). Text classification model based on fasttext. IEEE. https://doi.org/booktitle2020 IEEE International Conference on Artificial Intelligence and Information Systems (ICAIIS), 154–157.

Zhang, C., Tian, Y. X., & Hu, A. Y. (2025). Utilizing textual data from online reviews for daily tourism demand forecasting: A deep learning approach leveraging word embedding techniques. *Expert Systems with Applications*, *260*, 125439. 10.1016/j.eswa.2024.125439.